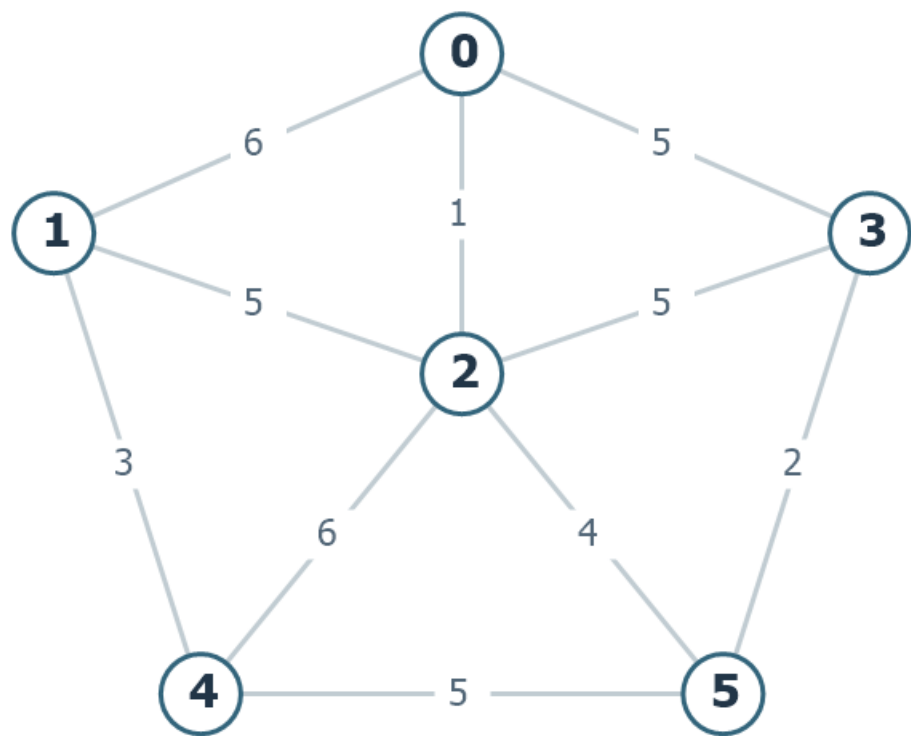


กราฟที่ 1 | ข้อมูลเริ่มต้น

กราฟไม่มีทิศทาง มีน้ำหนักบนเส้นเชื่อม และเริ่มการทำงานจากจุด 1



ข้อตกลงในการทำคำตอบ

DFS / BFS: เลือกเพื่อนบ้านเลขน้อยก่อน

DFS ใช้สแตก / BFS ใช้คิว

MST ใช้วิธี Prim เริ่มจากจุด 1

Shortest Path ใช้วิธี Dijkstra

น้ำหนักเส้นไม่มีผลต่อลำดับ DFS / BFS

ลำดับเยี่ยมชมอาจไม่ใช่เส้นทางต่อเนื่อง

กราฟที่ 1 | 1. Adjacency Matrix

แถวและคอลัมน์ใช้ลำดับจุดเดียวกัน: 0 = ไม่มีเส้นเชื่อม, ค่าอื่น = น้ำหนักเส้น

จุด	0	1	2	3	4	5
0	0	6	1	5	0	0
1	6	0	5	0	3	0
2	1	5	0	5	6	4
3	5	0	5	0	0	2
4	0	3	6	0	0	5
5	0	0	4	2	5	0

หากใช้เมทริกซ์แบบ 0 / 1 ให้แทนค่าน้ำหนักทุกค่าที่ไม่ใช่ 0 ด้วย 1

กราฟที่ 1 | 2. Adjacency List

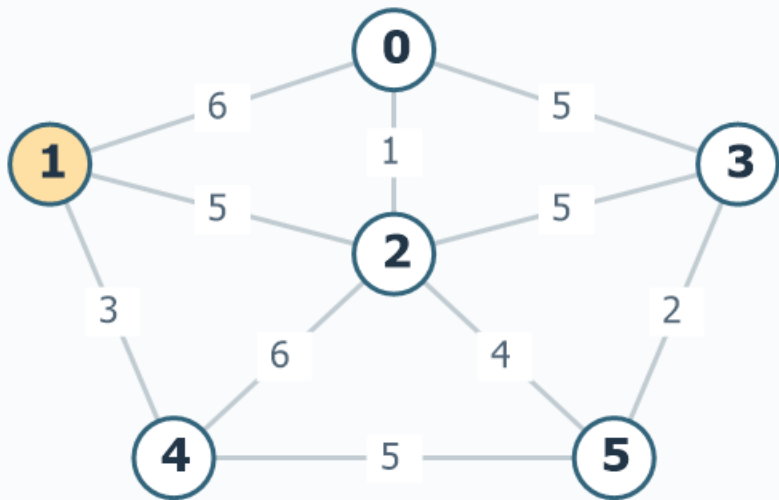
เขียนเป็น จุดข้างเคียง(น้ำหนัก) และเรียงเลขจุดจากน้อยไปมาก

จุด	รายการจุดข้างเคียง
0	1(6), 2(1), 3(5)
1	0(6), 2(5), 4(3)
2	0(1), 1(5), 3(5), 4(6), 5(4)
3	0(5), 2(5), 5(2)
4	1(3), 2(6), 5(5)
5	2(4), 3(2), 4(5)

กราฟที่ 1 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

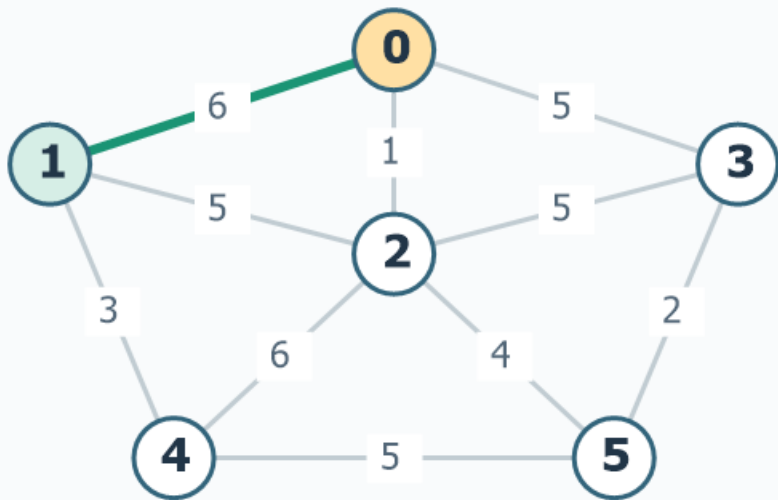
รอบที่ 1: เยี่ยมชมจุด 1



ลำดับที่ได้: 1
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 0
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1

รอบที่ 2: เยี่ยมชมจุด 0



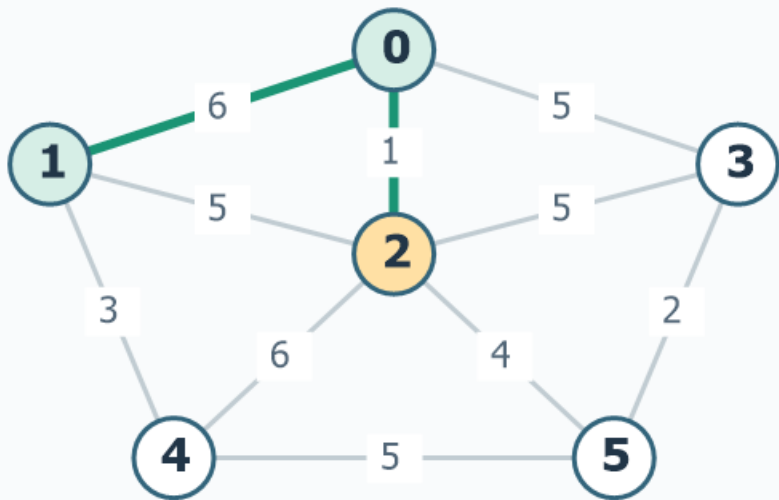
ลำดับที่ได้: 1 → 0
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 2
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	0
---	---

กราฟที่ 1 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

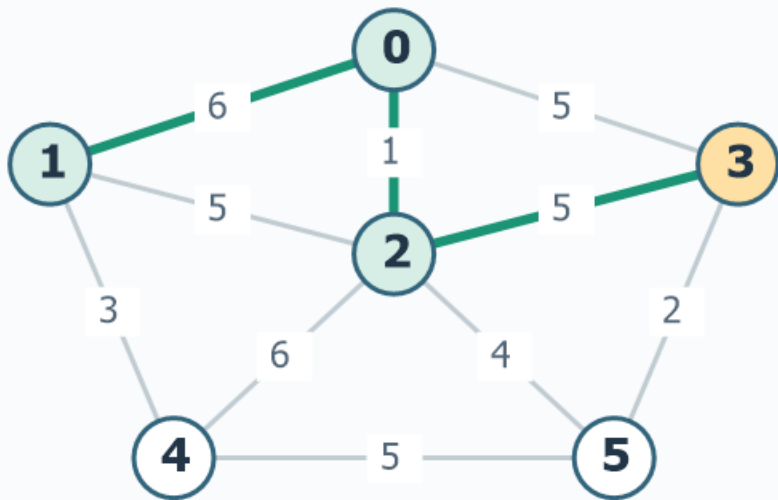
รอบที่ 3: เยี่ยมชมจุด 2



ลำดับที่ได้: $1 \rightarrow 0 \rightarrow 2$
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 3
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	0	2
---	---	---

รอบที่ 4: เยี่ยมชมจุด 3



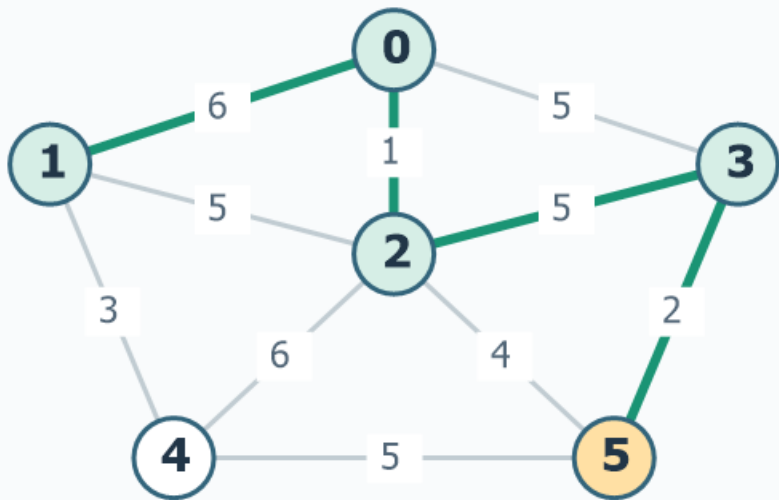
ลำดับที่ได้: $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 5
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	0	2	3
---	---	---	---

กราฟที่ 1 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

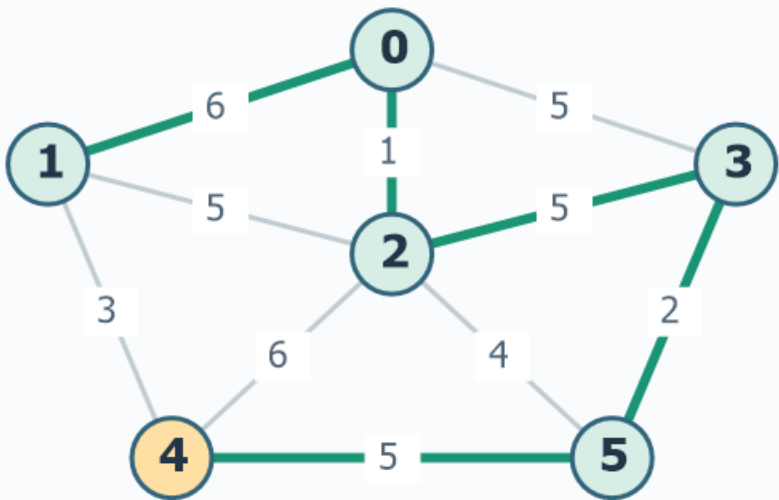
รอบที่ 5: เยี่ยมชมจุด 5



ลำดับที่ได้: 1 → 0 → 2 → 3 → 5
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 4
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	0	2	3	5
---	---	---	---	---

รอบที่ 6: เยี่ยมชมจุด 4

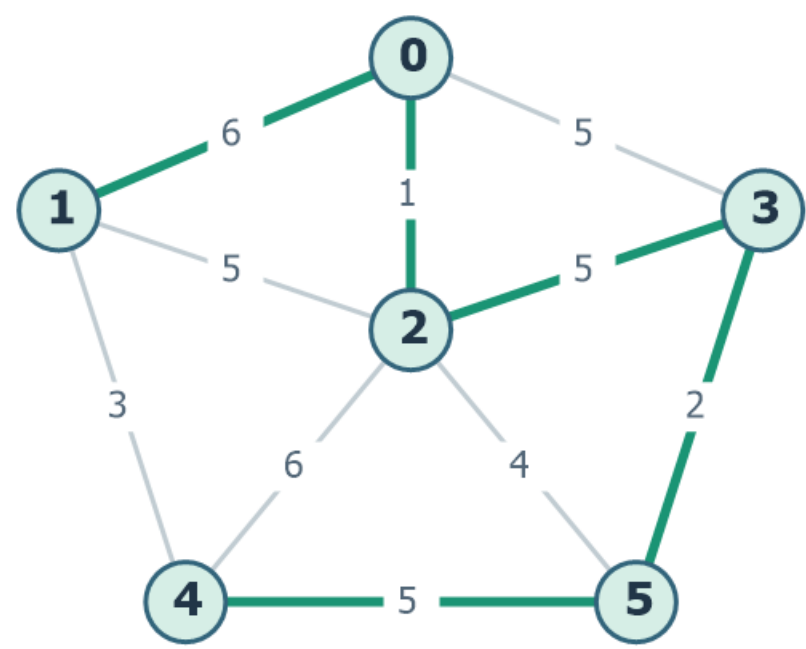


ลำดับที่ได้: 1 → 0 → 2 → 3 → 5 → 4
ไม่มีเพื่อนบ้านใหม่: เริ่มย้อนกลับ
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	0	2	3	5	4
---	---	---	---	---	---

กราฟที่ 1 | DFS: ย้อนกลับและสรุป

เมื่อไม่มีเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม ให้เอาจุดบนยอดสแตคออกและย้อนกลับ



ลำดับเยี่ยมชม

1 → 0 → 2 → 3 → 5 → 4

ลำดับย้อนกลับ

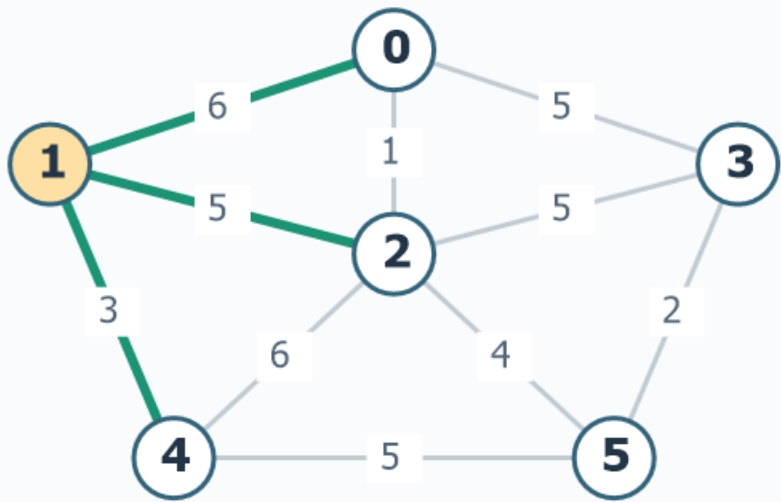
4 → 5
5 → 3
3 → 2
2 → 0
0 → 1

ท้ายสุดนำจุด 1 ออกจากสแตค: สแตคว่าง จบการทำงาน
เส้นสีเขียวเป็นต้นไม้ DFS ไม่ได้นำค่าน้ำหนักมาใช้ในการเลือกเส้น

กราฟที่ 1 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

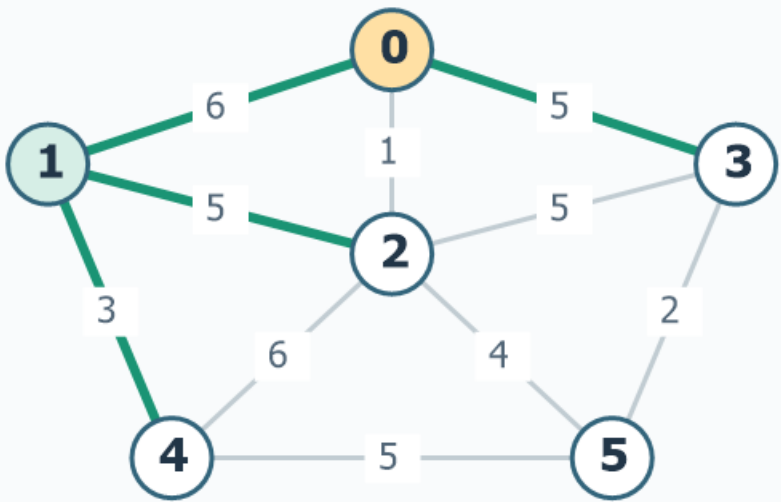
รอบที่ 1: นำจุด 1 ออกจากหัวคิว



เพิ่มท้ายคิว: 0, 2, 4
ลำดับที่ได้: 1
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

0	2	4
---	---	---

รอบที่ 2: นำจุด 0 ออกจากหัวคิว



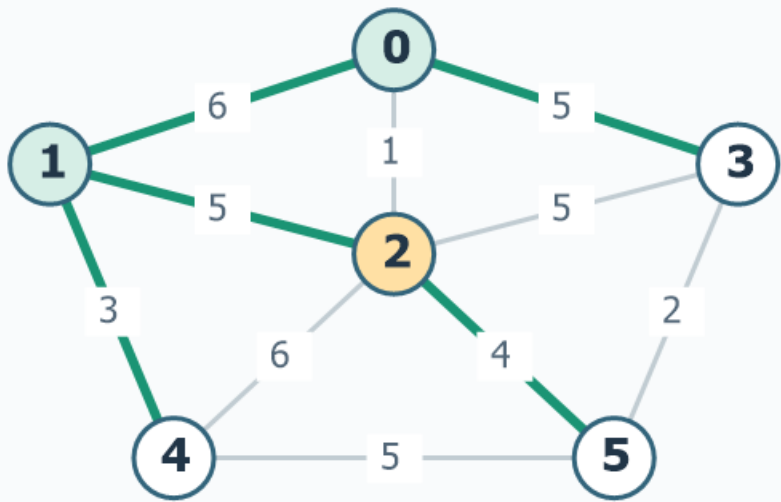
เพิ่มท้ายคิว: 3
ลำดับที่ได้: 1 → 0
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

2	4	3
---	---	---

กราฟที่ 1 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

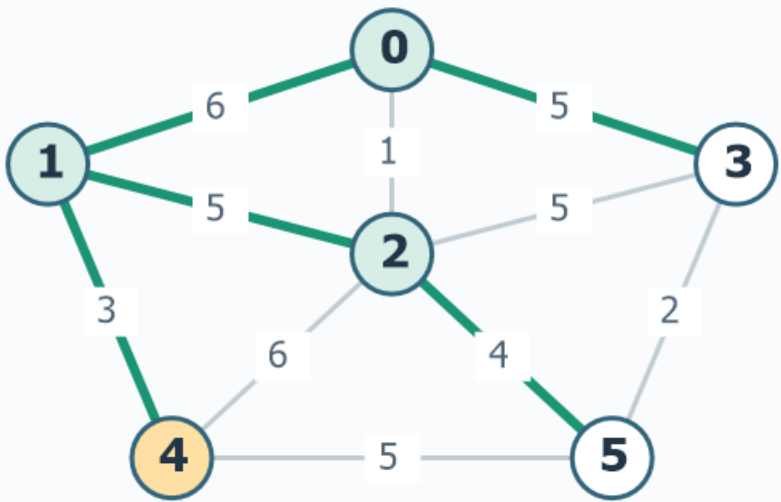
รอบที่ 3: นำจุด 2 ออกจากหัวคิว



เพิ่มท้ายคิว: 5
ลำดับที่ได้: 1 → 0 → 2
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

4	3	5
---	---	---

รอบที่ 4: นำจุด 4 ออกจากหัวคิว



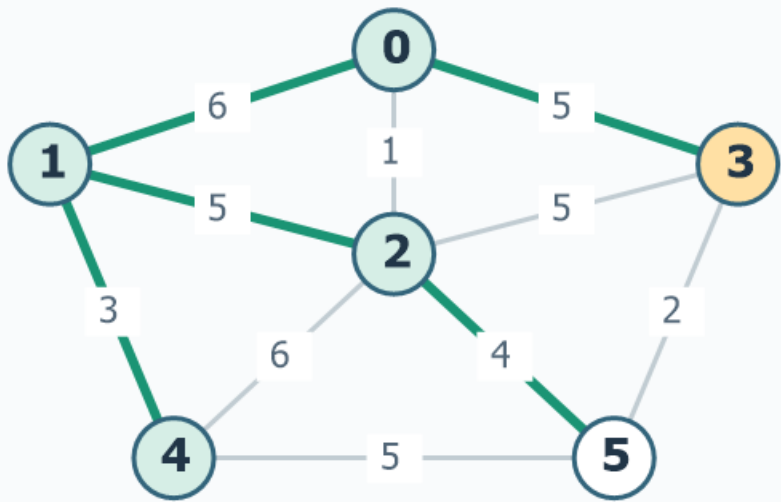
เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 0 → 2 → 4
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

3	5
---	---

กราฟที่ 1 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

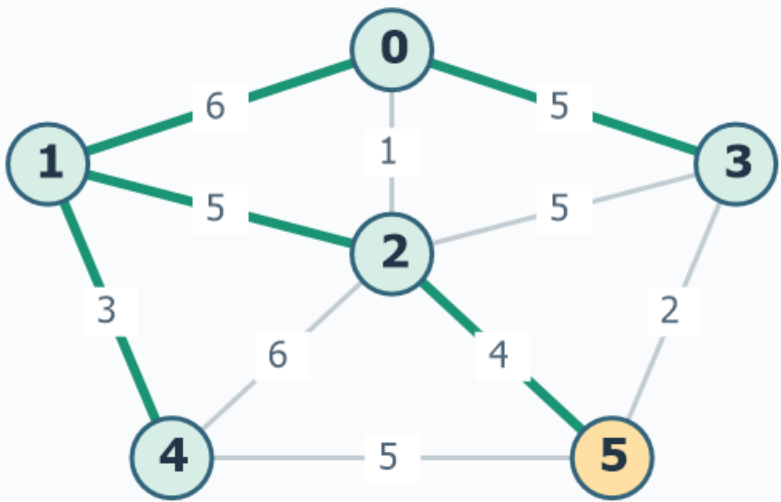
รอบที่ 5: นำจุด 3 ออกจากหัวคิว



เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 0 → 2 → 4 → 3
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

5

รอบที่ 6: นำจุด 5 ออกจากหัวคิว

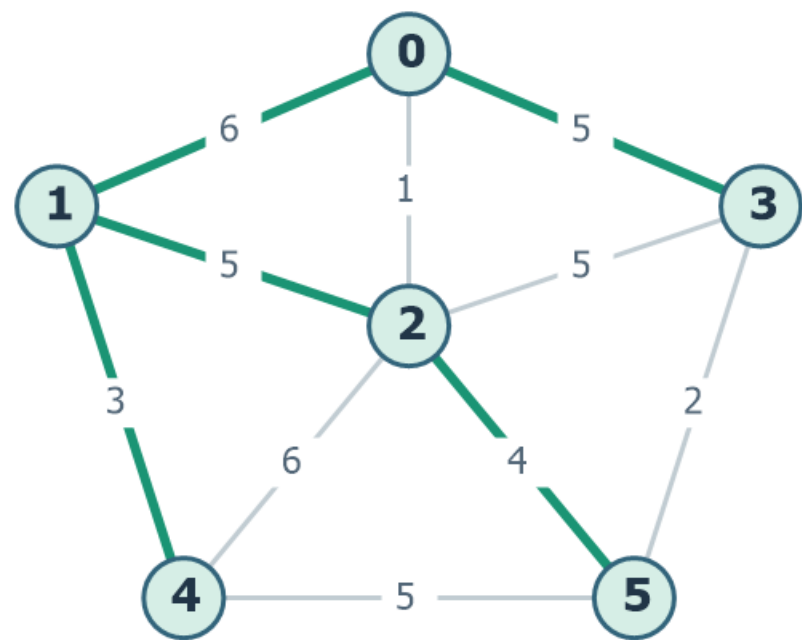


เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 0 → 2 → 4 → 3 → 5
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

ว่าง

กราฟที่ 1 | BFS: สรุปผล

กำหนดว่าเคยค้นพบแล้วทันทีที่เข้าคิว เพื่อไม่ให้ใส่จุดเดิมซ้ำ



ลำดับเยี่ยมชม

1 → 0 → 2 → 4 → 3 → 5

คิวว่าง: จบการทำงาน

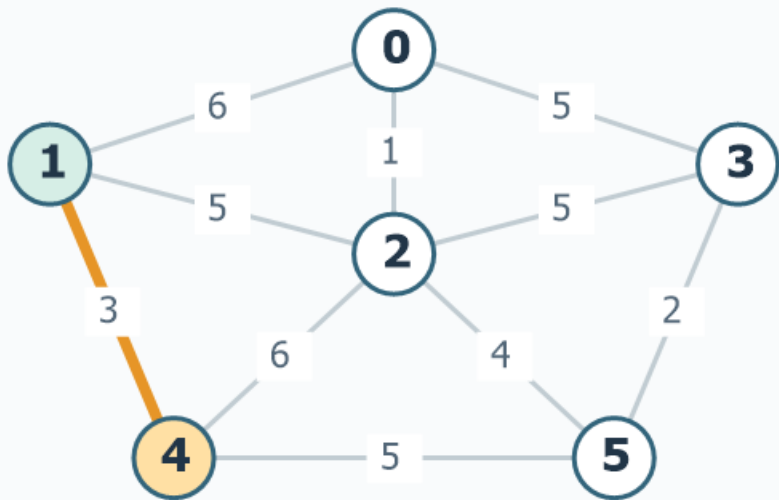
เส้นสีเขียวแสดงจุดที่ค้นพบจากแต่ละจุด เป็นต้นไม้ BFS

ลำดับเยี่ยมชมบอกการพบจุดครั้งแรก จุดติดกันในลำดับอาจไม่มีเส้นเชื่อม

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

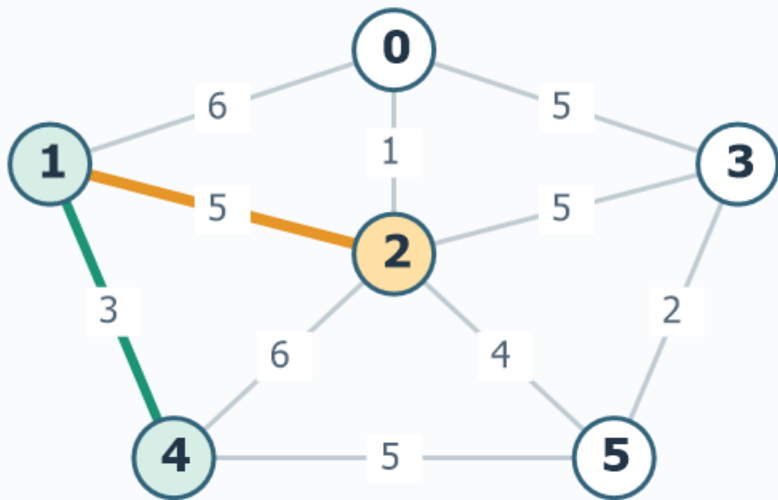
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้า

รอบที่ 1: เลือกเส้น 1-4 น้ำหนัก 3



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 4\}$
น้ำหนักสะสม = 3

รอบที่ 2: เลือกเส้น 1-2 น้ำหนัก 5

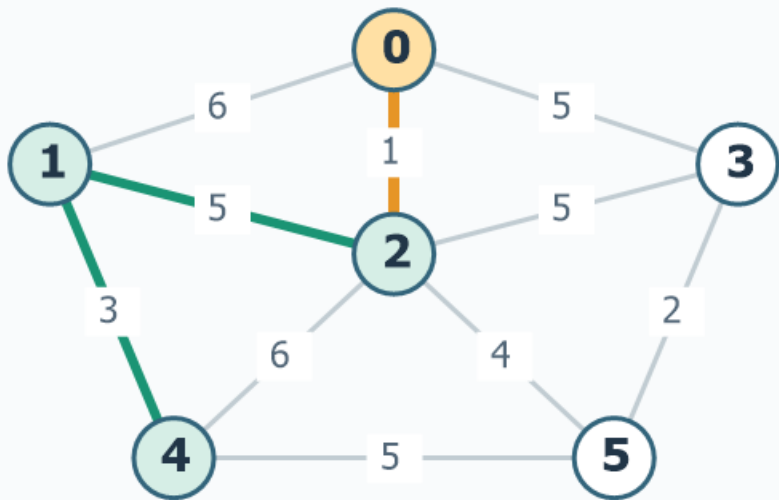


เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 2, 4\}$
น้ำหนักสะสม = 8

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

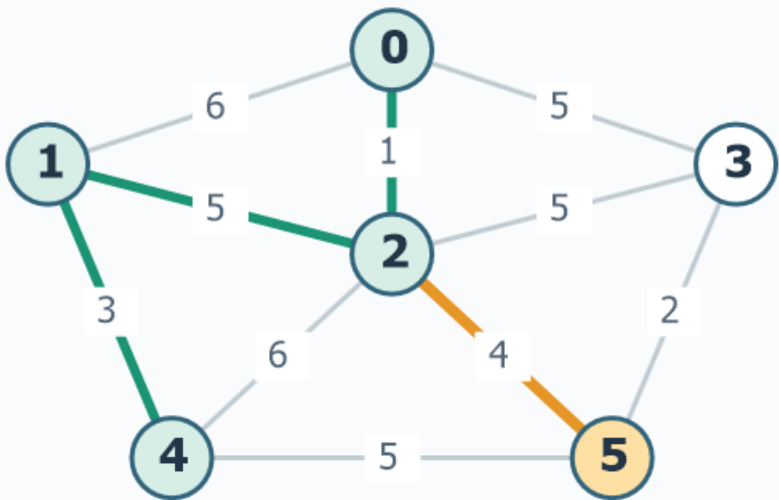
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้า

รอบที่ 3: เลือกเส้น 2-0 น้ำหนัก 1



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{0, 1, 2, 4\}$
น้ำหนักสะสม = 9

รอบที่ 4: เลือกเส้น 2-5 น้ำหนัก 4

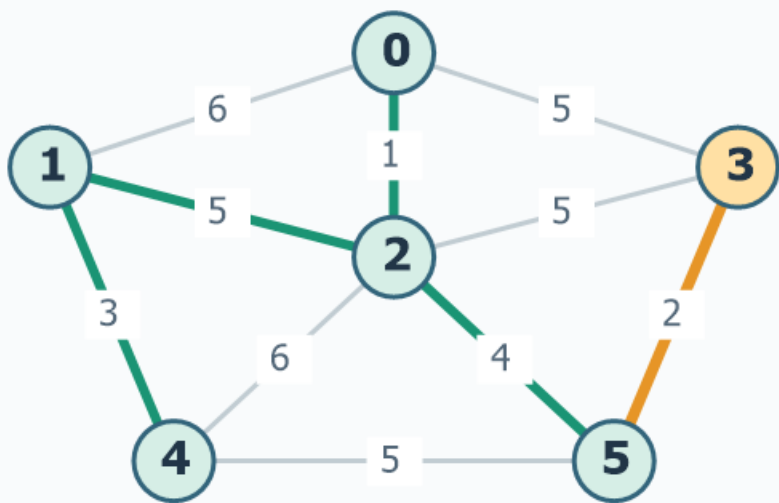


เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{0, 1, 2, 4, 5\}$
น้ำหนักสะสม = 13

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้านี้

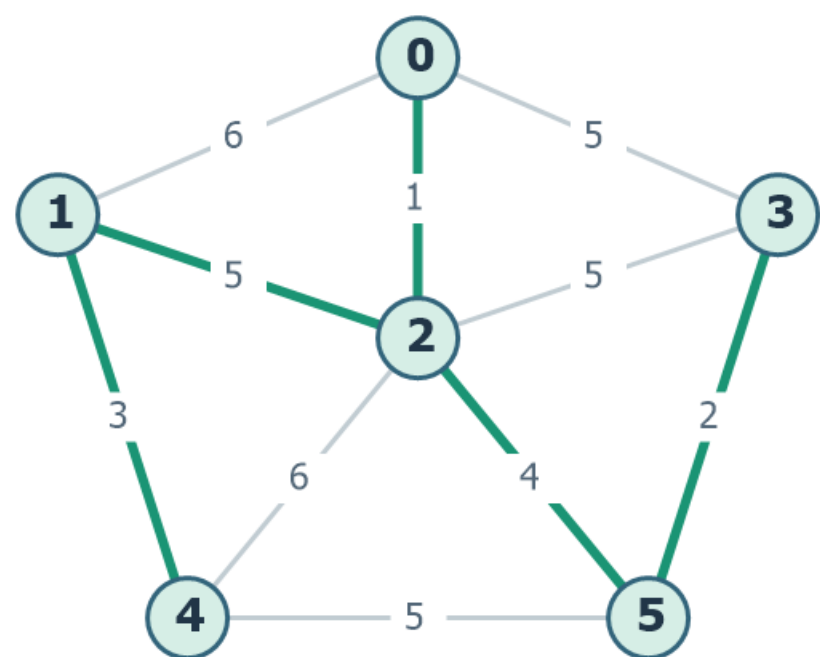
รอบที่ 5: เลือกเส้น 5-3 น้ำหนัก 2



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{0, 1, 2, 3, 4, 5\}$
น้ำหนักสะสม = 15

กราฟที่ 1 | Prim: สรุปร MST

เริ่ม $S = \{1\}$ | น้ำหนักเท่ากันเลือกคู่ปลายเส้นที่มีเลขน้อยก่อน | ไม่เลือกเส้นที่ทำให้เกิดวงจร



รอบ	เส้นที่เลือก	น้ำหนัก
1	1-4	3
2	1-2	5
3	2-0	1
4	2-5	4
5	5-3	2

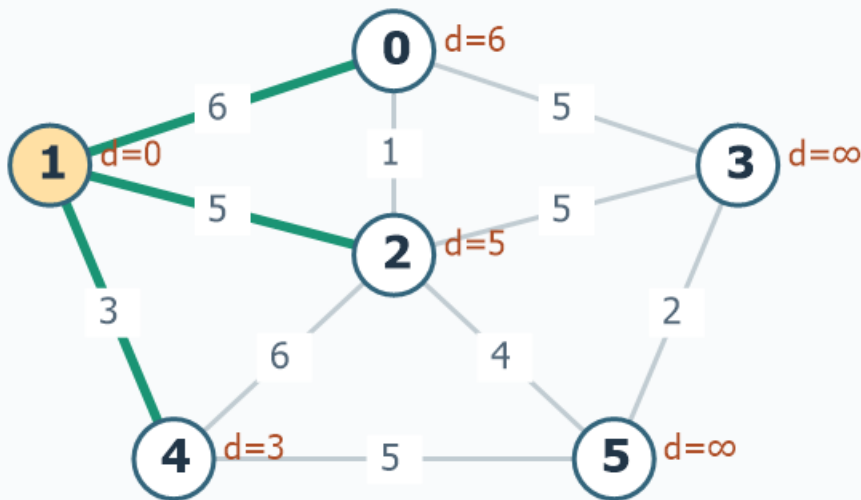
น้ำหนักรวม = 3 + 5 + 1 + 4 + 2 = 15

มี 6 จุด และ 5 เส้น เชื่อมครบทุกจุดโดยไม่มีวงจร

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

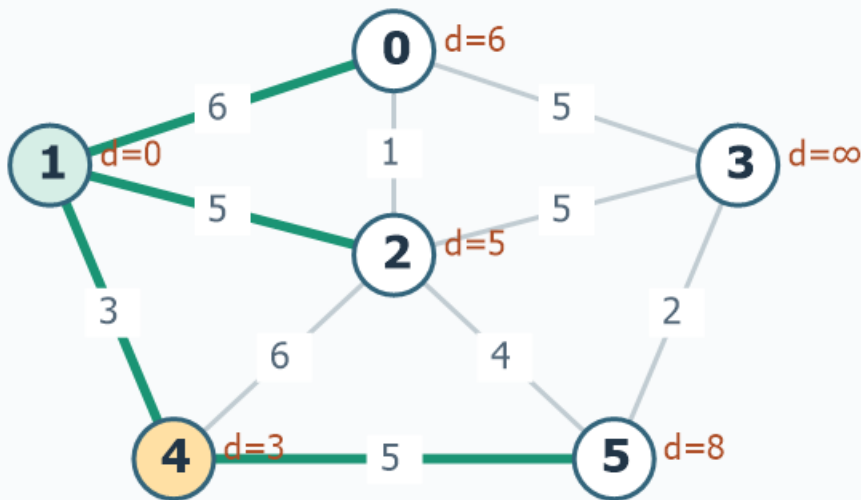
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 1: ยืนยันจุด 1, $d(1) = 0$



$d(0): \infty \rightarrow 0 + 6 = 6$
 $d(2): \infty \rightarrow 0 + 5 = 5$
 $d(4): \infty \rightarrow 0 + 3 = 3$

รอบที่ 2: ยืนยันจุด 4, $d(4) = 3$

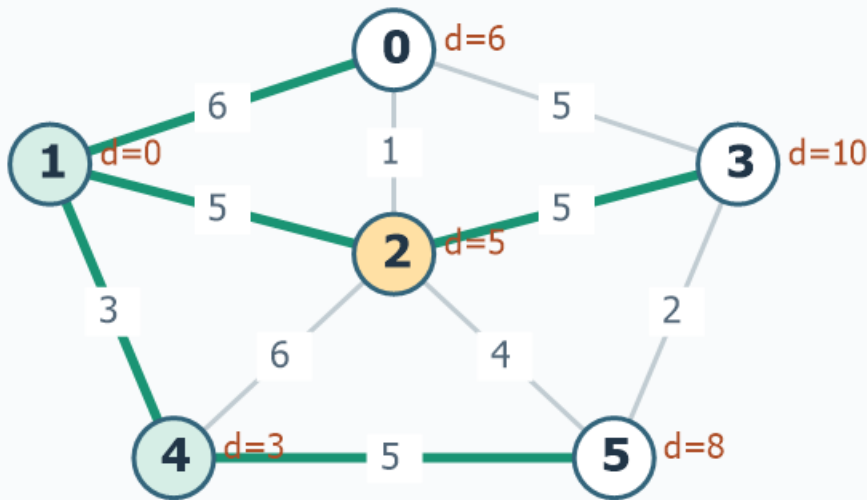


$d(5): \infty \rightarrow 3 + 5 = 8$

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

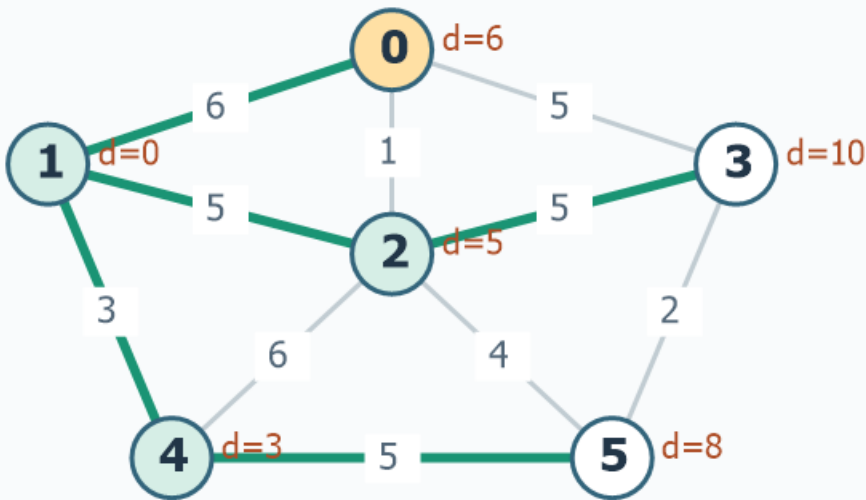
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 3: ยืนยันจุด 2, $d(2) = 5$



$d(3): \infty \rightarrow 5 + 5 = 10$

รอบที่ 4: ยืนยันจุด 0, $d(0) = 6$

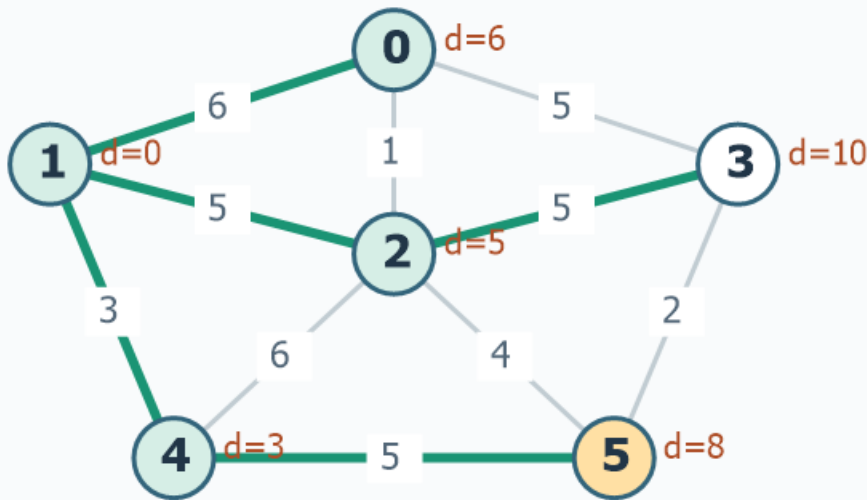


ไม่มีระยะทางที่ลดลงในรอบนี้

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

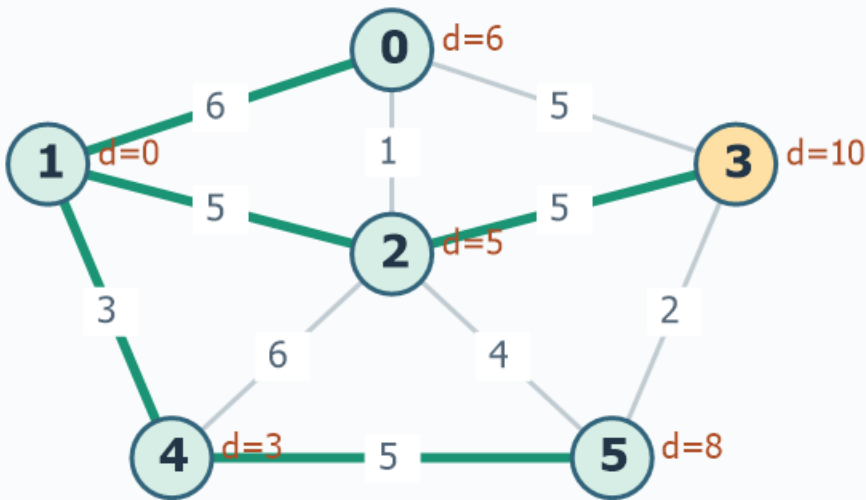
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 5: ยืนยันจุด 5, $d(5) = 8$



ไม่มีระยะทางที่ลดลงในรอบนี้

รอบที่ 6: ยืนยันจุด 3, $d(3) = 10$



ไม่มีระยะทางที่ลดลงในรอบนี้

กราฟที่ 1 | Dijkstra: ตารางระยะทางแต่ละรอบ

เริ่ม $d(1) = 0$, จุดอื่น $= \infty$ | ยืนยันจุดที่มีระยะทางน้อยสุด | ค่าเท่ากันเลือกจุดเลขน้อยก่อน

รอบ	ยืนยัน	d(0)	d(1)	d(2)	d(3)	d(4)	d(5)
1	1	6	0	5	∞	3	∞
2	4	6	0	5	∞	3	8
3	2	6	0	5	10	3	8
4	0	6	0	5	10	3	8
5	5	6	0	5	10	3	8
6	3	6	0	5	10	3	8

ปรับระยะทางเมื่อ $d(\text{จุดปัจจุบัน}) + \text{น้ำหนักเส้น} < \text{ระยะทางเดิมเท่านั้น}$

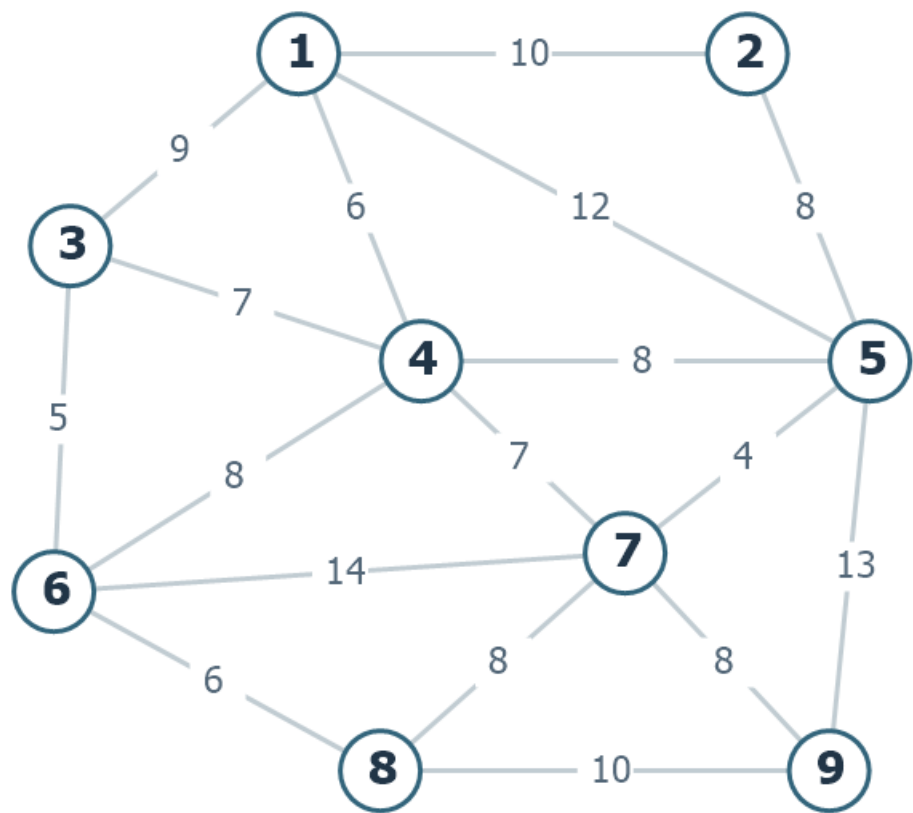
กราฟที่ 1 | Dijkstra: เส้นทางสั้นที่สุดจากจุด 1

เมื่อระยะทางใหม่เท่าค่าเดิม ให้เก็บจุดก่อนหน้าเดิมไว้ จึงได้คำตอบตามเส้นทางด้านล่าง

ปลายทาง	เส้นทางจากจุด 1	ระยะทาง
0	1 → 0	6
1	1	0
2	1 → 2	5
3	1 → 2 → 3	10
4	1 → 4	3
5	1 → 4 → 5	8

กราฟที่ 2 | ข้อมูลเริ่มต้น

กราฟไม่มีทิศทาง มีน้ำหนักบนเส้นเชื่อม และเริ่มการทำงานจากจุด 1



ข้อตกลงในการทำคำตอบ

DFS / BFS: เลือกเพื่อนบ้านเลขน้อยก่อน

DFS ใช้สแตก / BFS ใช้คิว

MST ใช้วิธี Prim เริ่มจากจุด 1

Shortest Path ใช้วิธี Dijkstra

น้ำหนักเส้นไม่มีผลต่อลำดับ DFS / BFS

ลำดับเยี่ยมชมอาจไม่ใช่เส้นทางต่อเนื่อง

กราฟที่ 2 | 1. Adjacency Matrix

แถวและคอลัมน์ใช้ลำดับจุดเดียวกัน: 0 = ไม่มีเส้นเชื่อม, ค่าอื่น = น้ำหนักเส้น

จุด	1	2	3	4	5	6	7	8	9
1	0	10	9	6	12	0	0	0	0
2	10	0	0	0	8	0	0	0	0
3	9	0	0	7	0	5	0	0	0
4	6	0	7	0	8	8	7	0	0
5	12	8	0	8	0	0	4	0	13
6	0	0	5	8	0	0	14	6	0
7	0	0	0	7	4	14	0	8	8
8	0	0	0	0	0	6	8	0	10
9	0	0	0	0	13	0	8	10	0

หากใช้เมทริกซ์แบบ 0 / 1 ให้แทนค่าน้ำหนักทุกค่าที่ไม่ใช่ 0 ด้วย 1

กราฟที่ 2 | 2. Adjacency List

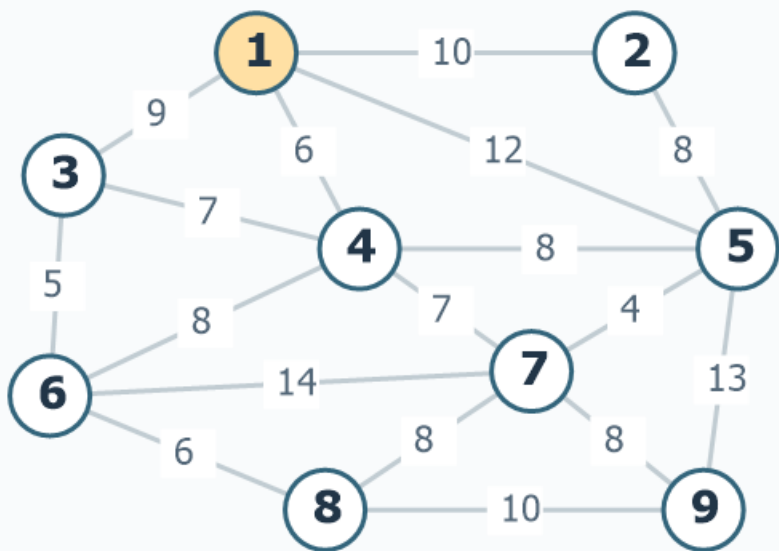
เขียนเป็น จุดข้างเคียง(น้ำหนัก) และเรียงเลขจุดจากน้อยไปมาก

จุด	รายการจุดข้างเคียง
1	2(10), 3(9), 4(6), 5(12)
2	1(10), 5(8)
3	1(9), 4(7), 6(5)
4	1(6), 3(7), 5(8), 6(8), 7(7)
5	1(12), 2(8), 4(8), 7(4), 9(13)
6	3(5), 4(8), 7(14), 8(6)
7	4(7), 5(4), 6(14), 8(8), 9(8)
8	6(6), 7(8), 9(10)
9	5(13), 7(8), 8(10)

กราฟที่ 2 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

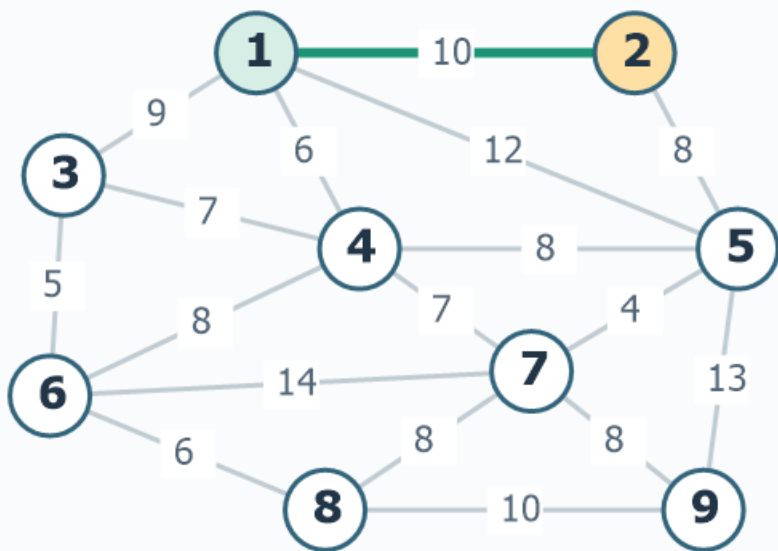
รอบที่ 1: เยี่ยมชมจุด 1



ลำดับที่ได้: 1
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 2
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1

รอบที่ 2: เยี่ยมชมจุด 2



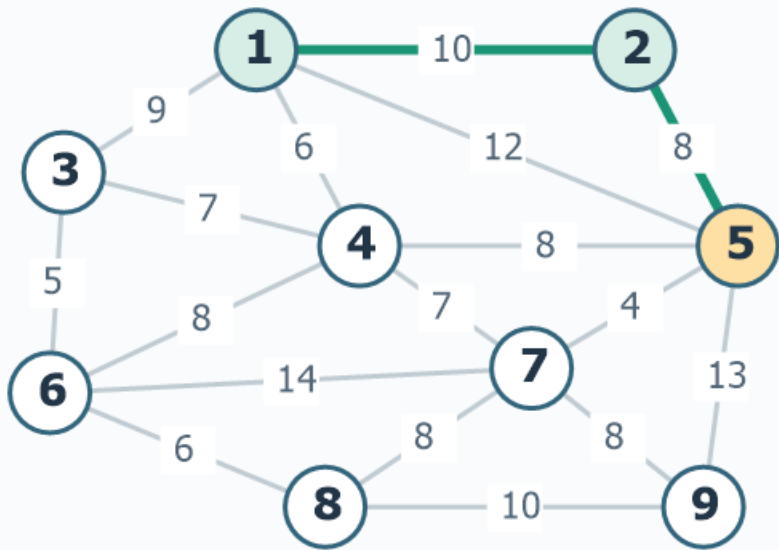
ลำดับที่ได้: 1 → 2
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 5
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1 2

กราฟที่ 2 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

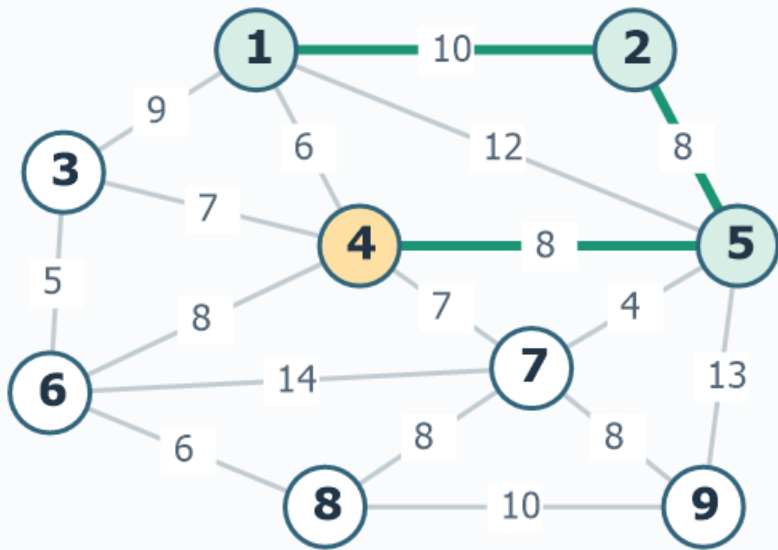
รอบที่ 3: เยี่ยมชมจุด 5



ลำดับที่ได้: 1 → 2 → 5
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 4
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5
---	---	---

รอบที่ 4: เยี่ยมชมจุด 4



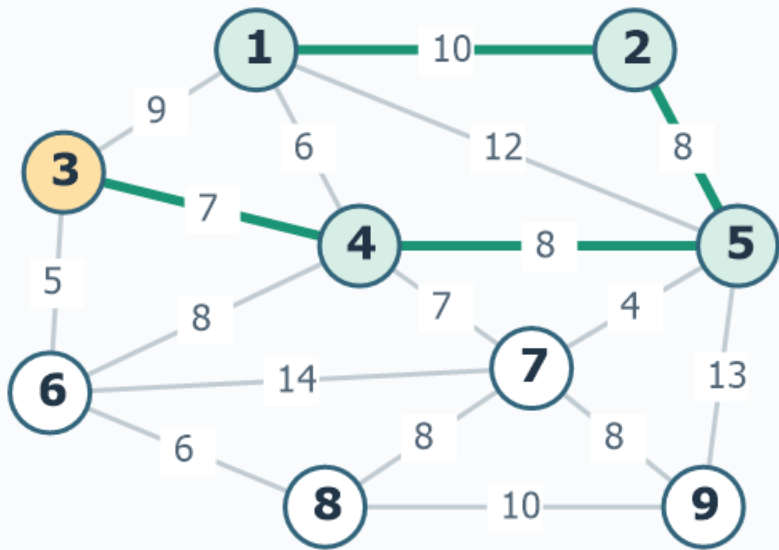
ลำดับที่ได้: 1 → 2 → 5 → 4
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 3
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4
---	---	---	---

กราฟที่ 2 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

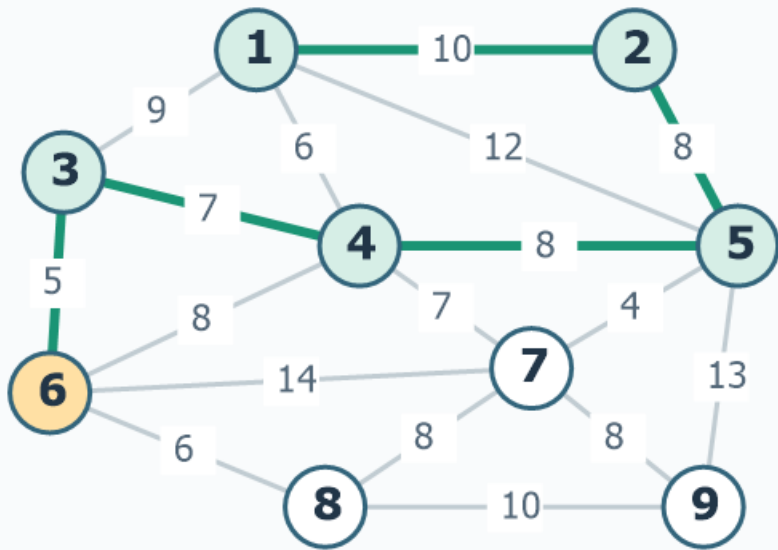
รอบที่ 5: เยี่ยมชมจุด 3



ลำดับที่ได้: 1 → 2 → 5 → 4 → 3
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 6
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4	3
---	---	---	---	---

รอบที่ 6: เยี่ยมชมจุด 6



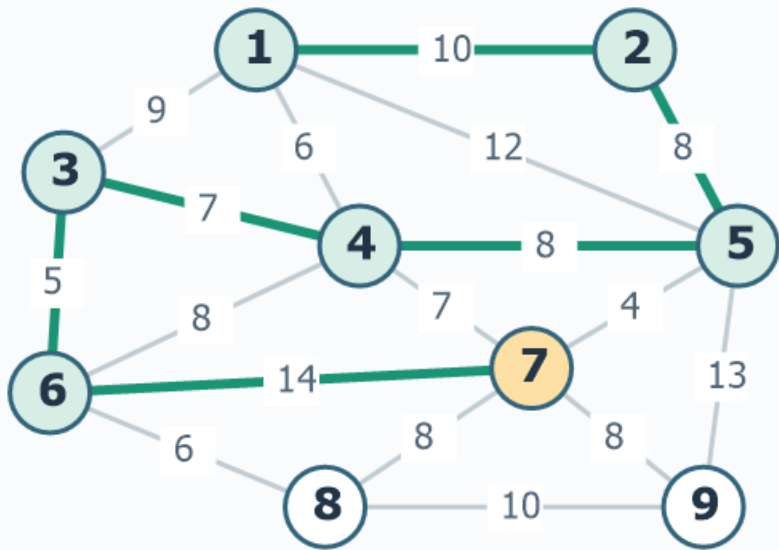
ลำดับที่ได้: 1 → 2 → 5 → 4 → 3 → 6
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 7
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4	3	6
---	---	---	---	---	---

กราฟที่ 2 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

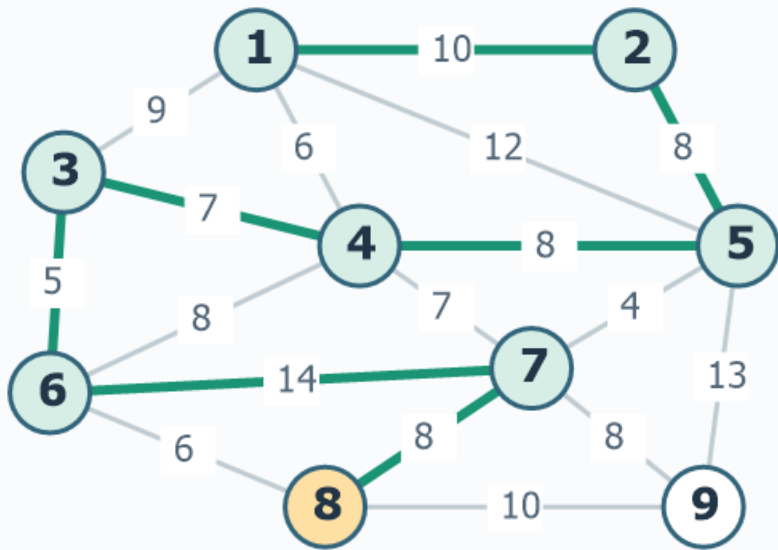
รอบที่ 7: เยี่ยมชมจุด 7



ลำดับที่ได้: 1 → 2 → 5 → 4 → 3 → 6 → 7
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 8
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4	3	6	7
---	---	---	---	---	---	---

รอบที่ 8: เยี่ยมชมจุด 8



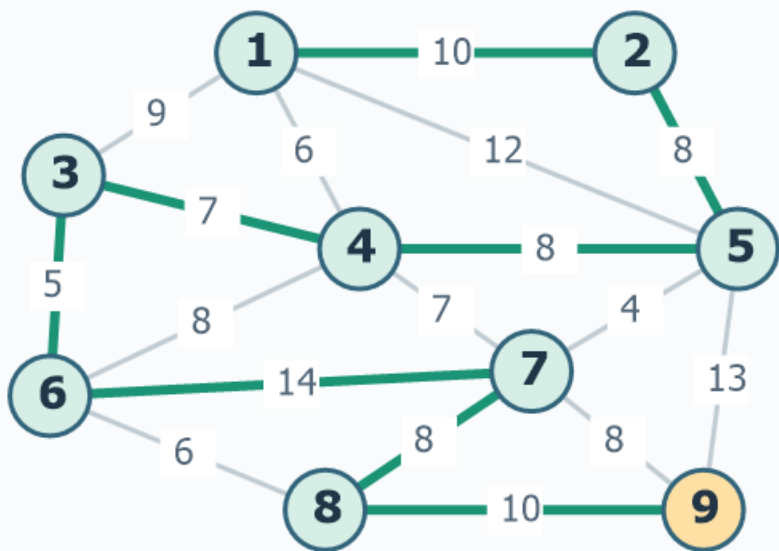
ลำดับที่ได้: 1 → 2 → 5 → 4 → 3 → 6 → 7 → 8
เลือกเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม: 9
สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4	3	6	7	8
---	---	---	---	---	---	---	---

กราฟที่ 2 | 3. Depth-First Search (DFS)

เริ่มสแตกว่าง แล้วเข้าจุด 1 | จุดสีส้ม = จุดปัจจุบัน | เส้นสีเขียว = เส้นที่ใช้ค้นพบจุด

รอบที่ 9: เยี่ยมชมจุด 9



ลำดับที่ได้: 1 → 2 → 5 → 4 → 3 → 6 → 7 → 8 → 9

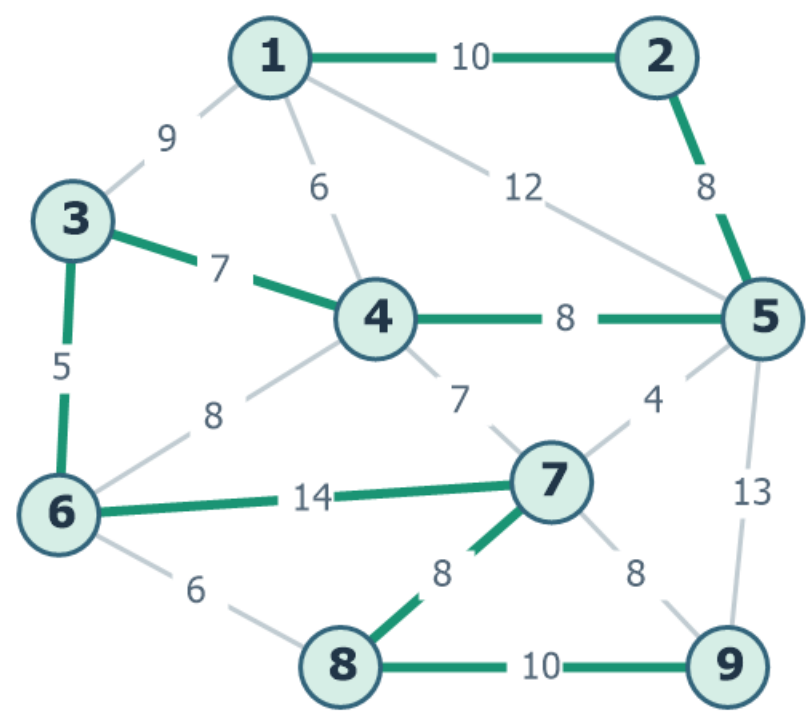
ไม่มีเพื่อนบ้านใหม่: เริ่มย้อนกลับ

สแตกหลังเข้าจุด (ยอดสแตกอยู่ขวา)

1	2	5	4	3	6	7	8	9
---	---	---	---	---	---	---	---	---

กราฟที่ 2 | DFS: ย้อนกลับและสรุป

เมื่อไม่มีเพื่อนบ้านที่ยังไม่เคยเยี่ยมชม ให้เอาจุดบนยอดสแตคออกและย้อนกลับ



ลำดับเยี่ยมชม

1 → 2 → 5 → 4 → 3 → 6 → 7 → 8 → 9

ลำดับย้อนกลับ

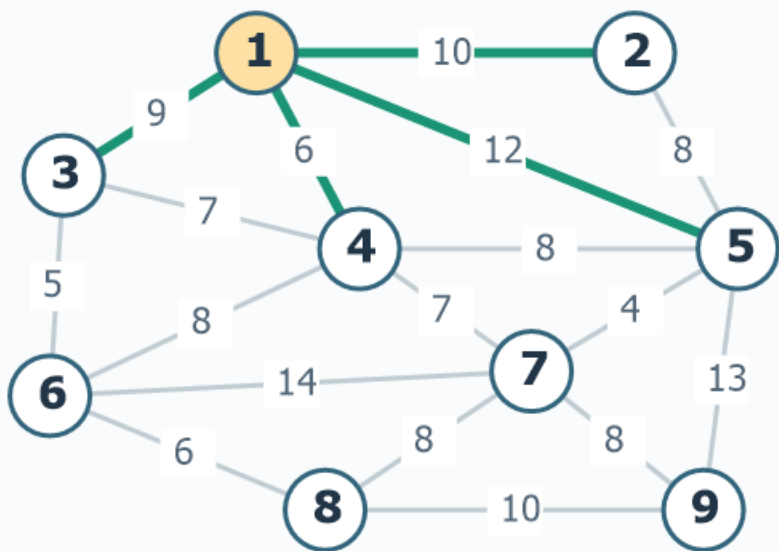
9 → 8
8 → 7
7 → 6
6 → 3
3 → 4
4 → 5
5 → 2
2 → 1

ท้ายสุดนำจุด 1 ออกจากสแตค: สแตคว่าง จบการทำงาน
เส้นสีเขียวเป็นต้นไม้ DFS ไม่ได้นำค่าน้ำหนักมาใช้ในการเลือกเส้น

กราฟที่ 2 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

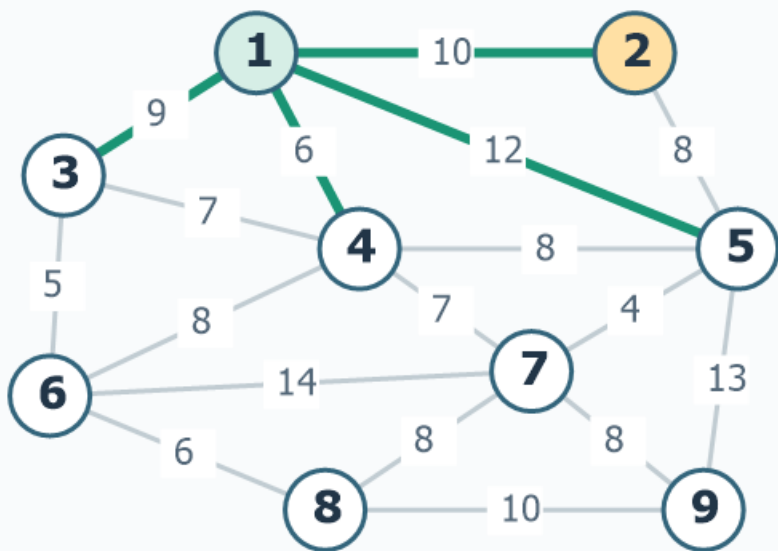
รอบที่ 1: นำจุด 1 ออกจากหัวคิว



เพิ่มท้ายคิว: 2, 3, 4, 5
ลำดับที่ได้: 1
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

2	3	4	5
---	---	---	---

รอบที่ 2: นำจุด 2 ออกจากหัวคิว



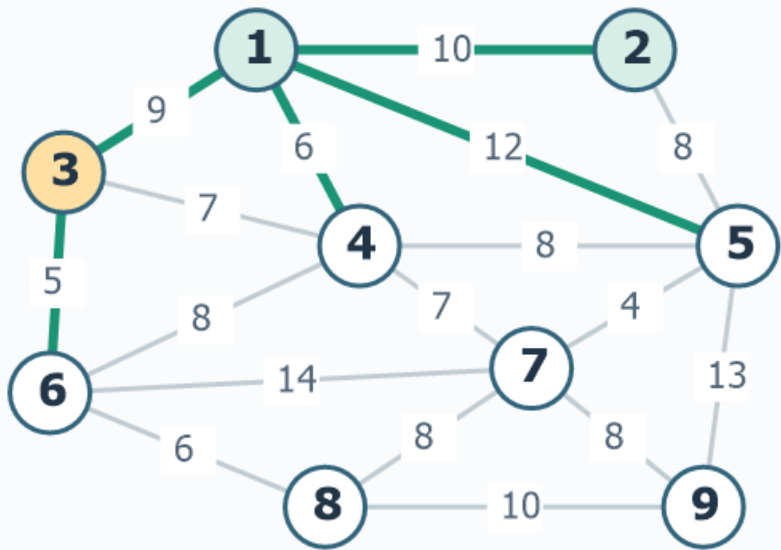
เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 2
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

3	4	5
---	---	---

กราฟที่ 2 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

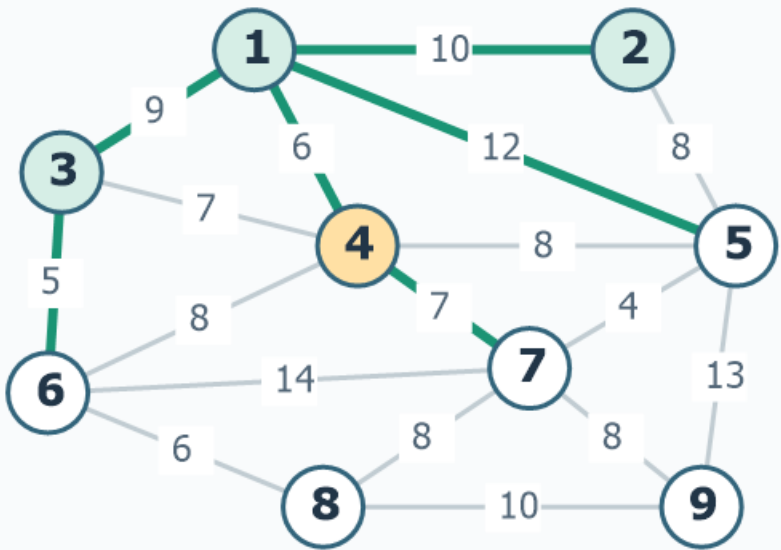
รอบที่ 3: นำจุด 3 ออกจากหัวคิว



เพิ่มท้ายคิว: 6
ลำดับที่ได้: 1 → 2 → 3
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

4	5	6
---	---	---

รอบที่ 4: นำจุด 4 ออกจากหัวคิว



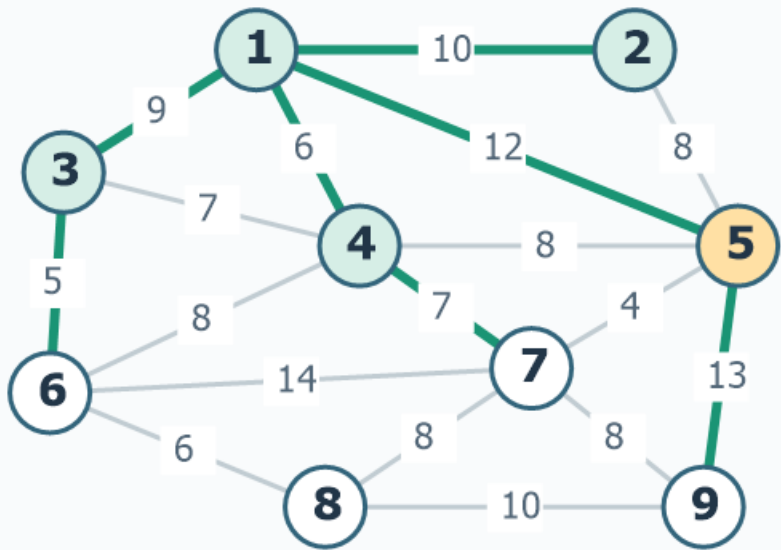
เพิ่มท้ายคิว: 7
ลำดับที่ได้: 1 → 2 → 3 → 4
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

5	6	7
---	---	---

กราฟที่ 2 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

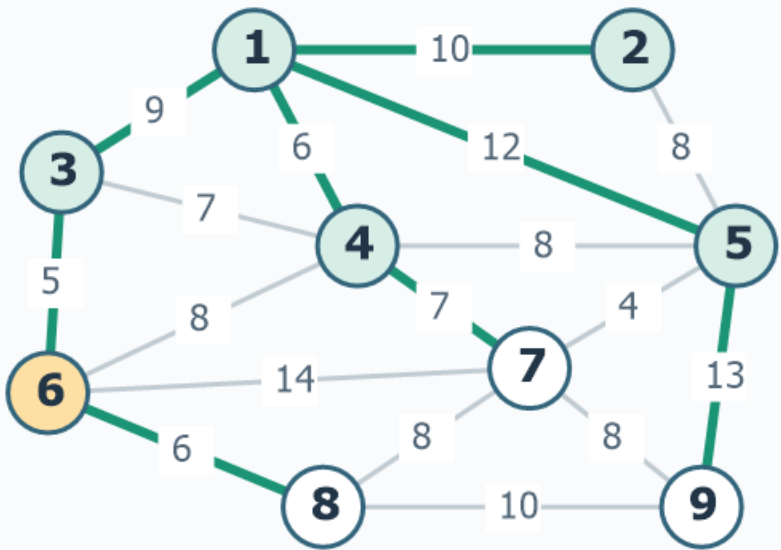
รอบที่ 5: นำจุด 5 ออกจากหัวคิว



เพิ่มท้ายคิว: 9
ลำดับที่ได้: 1 → 2 → 3 → 4 → 5
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

6	7	9
---	---	---

รอบที่ 6: นำจุด 6 ออกจากหัวคิว



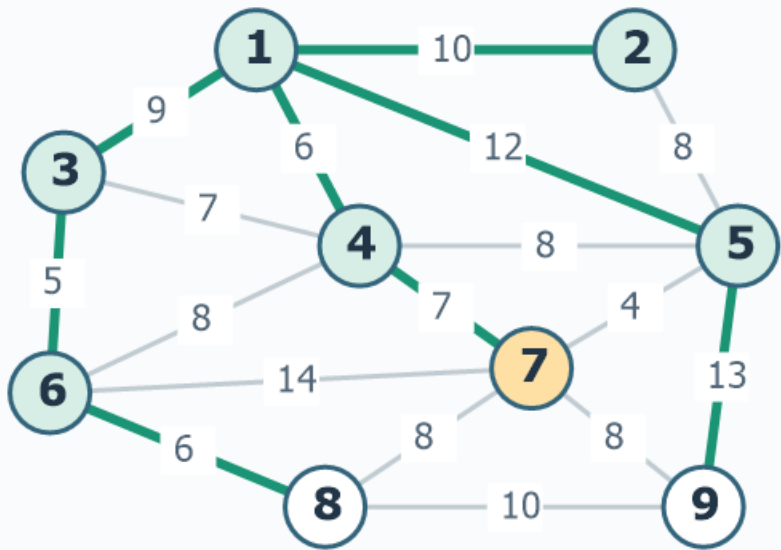
เพิ่มท้ายคิว: 8
ลำดับที่ได้: 1 → 2 → 3 → 4 → 5 → 6
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

7	9	8
---	---	---

กราฟที่ 2 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

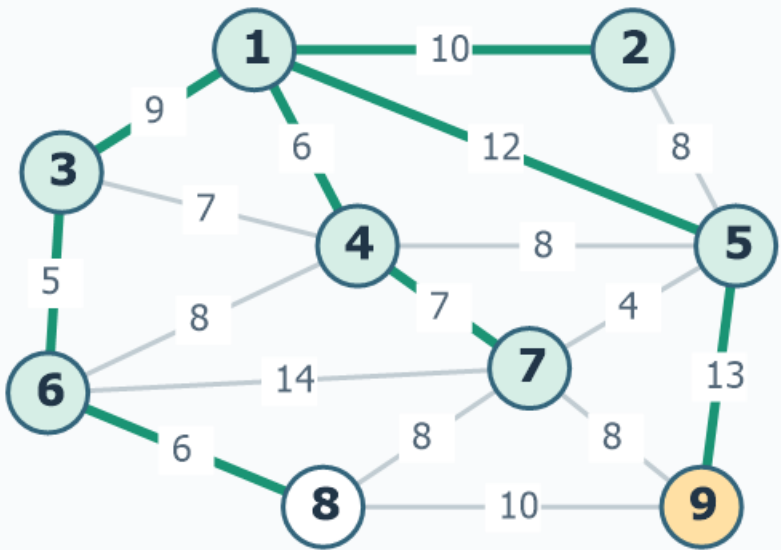
รอบที่ 7: นำจุด 7 ออกจากหัวคิว



เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 2 → 3 → 4 → 5 → 6 → 7
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

9	8
---	---

รอบที่ 8: นำจุด 9 ออกจากหัวคิว



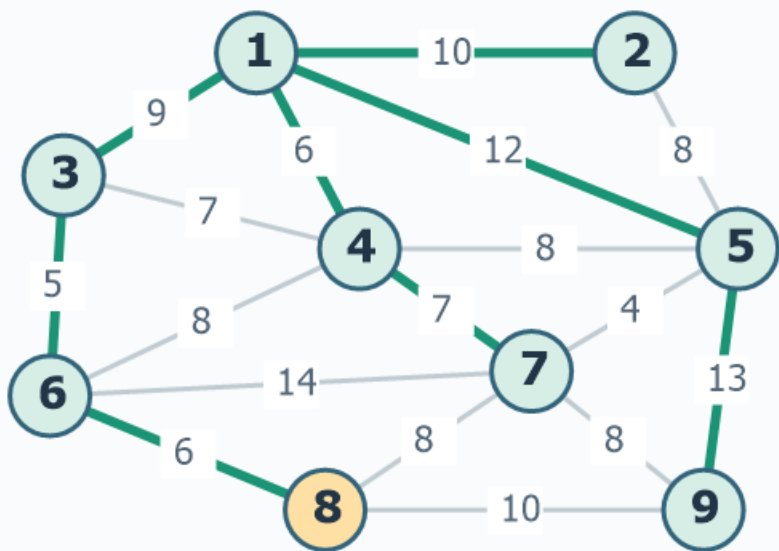
เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 9
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)

8

กราฟที่ 2 | 4. Breadth-First Search (BFS)

เริ่มคิว [1] | จุดสีส้ม = จุดปัจจุบัน | จุดสีเขียว = ประมวลผลแล้ว | เส้นสีเขียว = เส้นค้นพบ

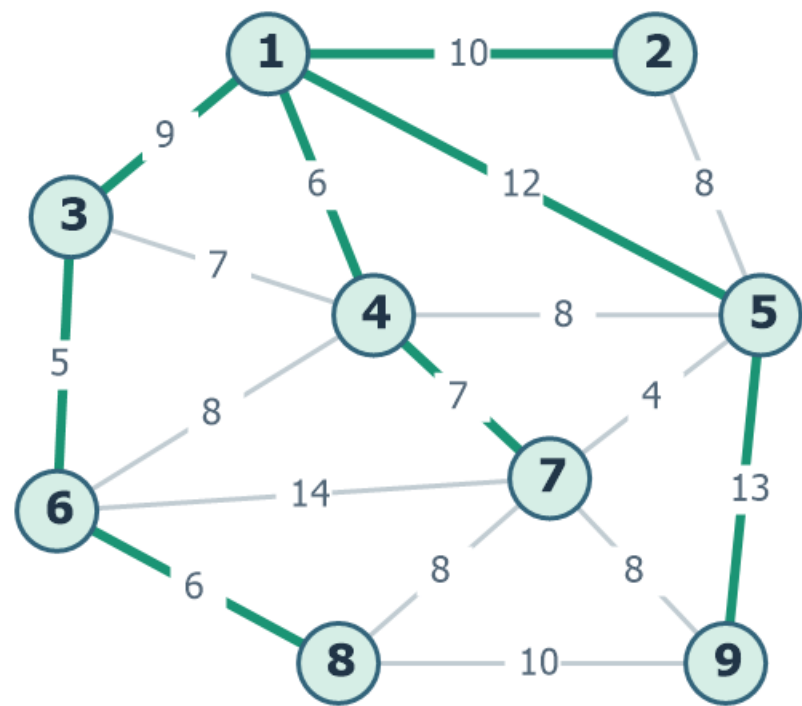
รอบที่ 9: นำจุด 8 ออกจากหัวคิว



เพิ่มท้ายคิว: ไม่มีจุดใหม่
ลำดับที่ได้: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 9 → 8
คิวหลังจบรอบ (หัวคิวอยู่ซ้าย)
ว่าง

กราฟที่ 2 | BFS: สรุปผล

กำหนดว่าเคยค้นพบแล้วทันทีที่เข้าคิว เพื่อไม่ให้ใส่จุดเดิมซ้ำ



ลำดับเยี่ยมชม

1 → 2 → 3 → 4 → 5 → 6 → 7 → 9 → 8

คิวว่าง: จบการทำงาน

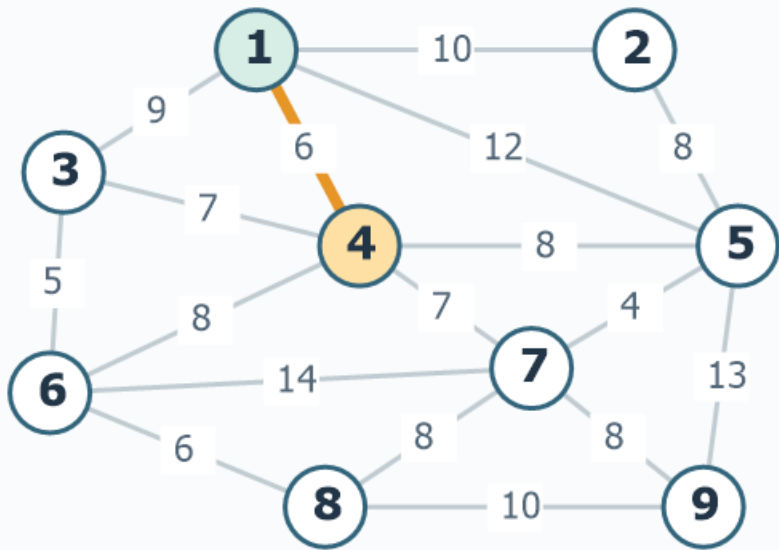
เส้นสีเขียวแสดงจุดที่ค้นพบจากแต่ละจุด เป็นต้นไม้ BFS

ลำดับเยี่ยมชมบอกการพบจุดครั้งแรก จุดติดกันในลำดับอาจไม่มีเส้นเชื่อม

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

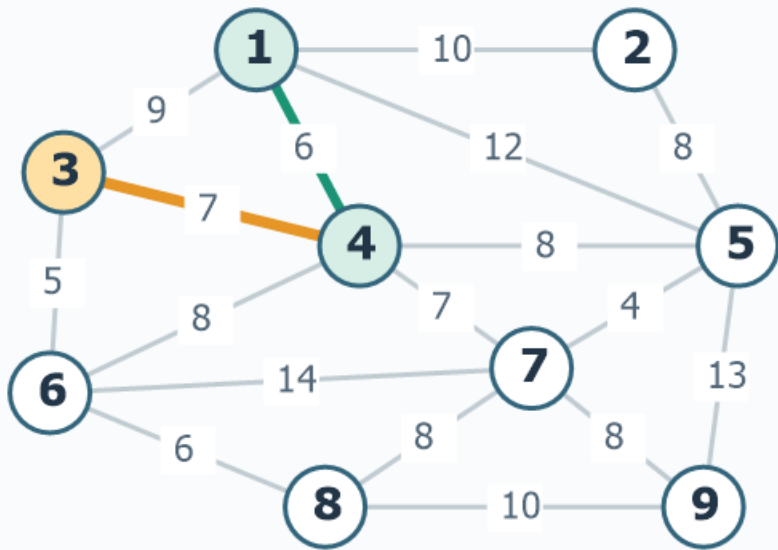
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้า

รอบที่ 1: เลือกเส้น 1-4 น้ำหนัก 6



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 4\}$
น้ำหนักสะสม = 6

รอบที่ 2: เลือกเส้น 4-3 น้ำหนัก 7

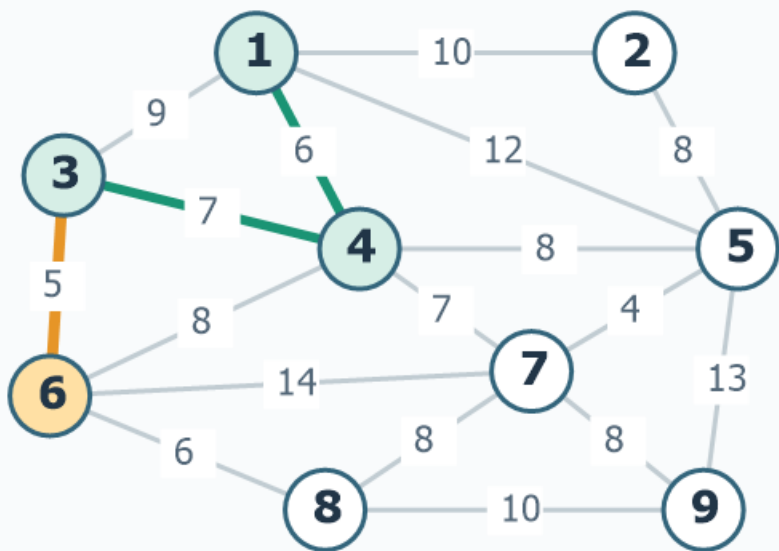


เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 3, 4\}$
น้ำหนักสะสม = 13

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

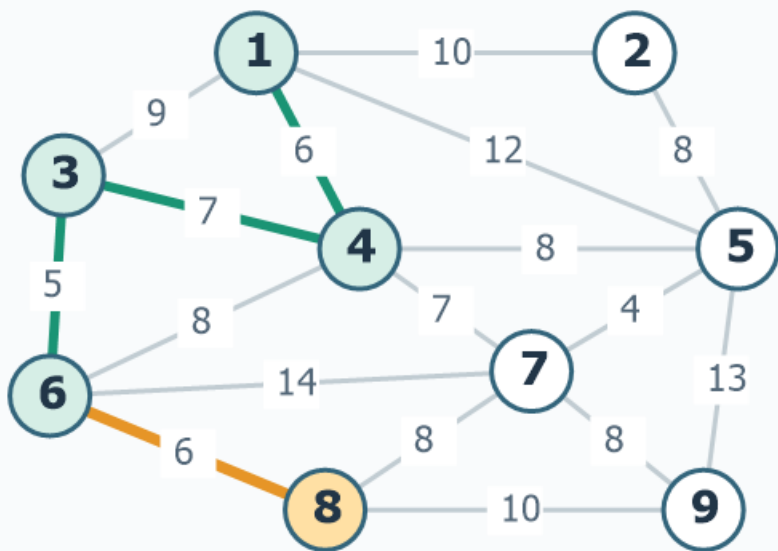
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้านี้

รอบที่ 3: เลือกเส้น 3-6 น้ำหนัก 5



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 3, 4, 6\}$
น้ำหนักสะสม = 18

รอบที่ 4: เลือกเส้น 6-8 น้ำหนัก 6

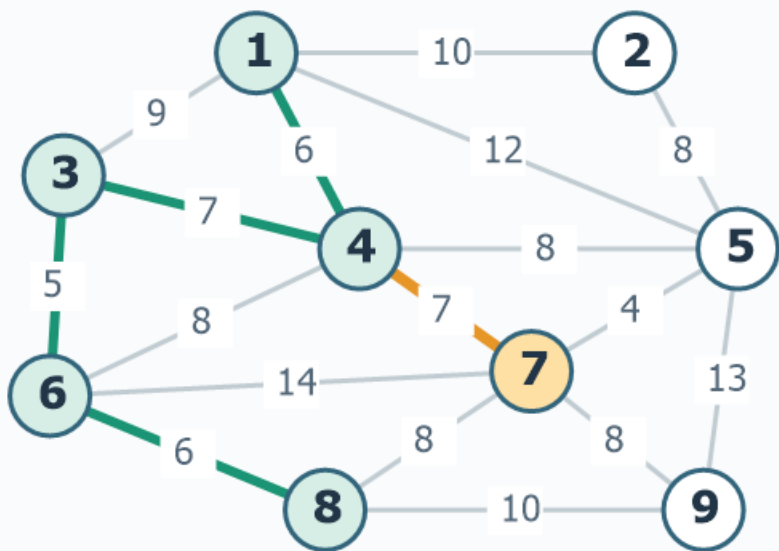


เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 3, 4, 6, 8\}$
น้ำหนักสะสม = 24

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

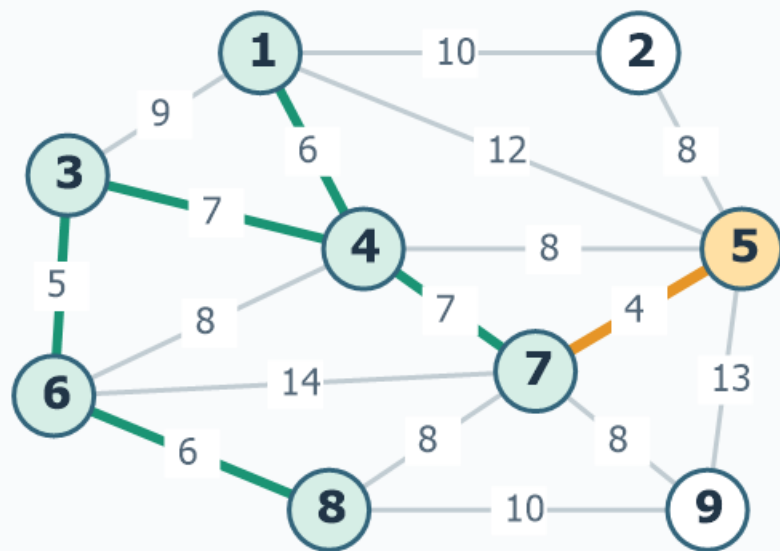
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้า

รอบที่ 5: เลือกเส้น 4-7 น้ำหนัก 7



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 3, 4, 6, 7, 8\}$
น้ำหนักสะสม = 31

รอบที่ 6: เลือกเส้น 7-5 น้ำหนัก 4

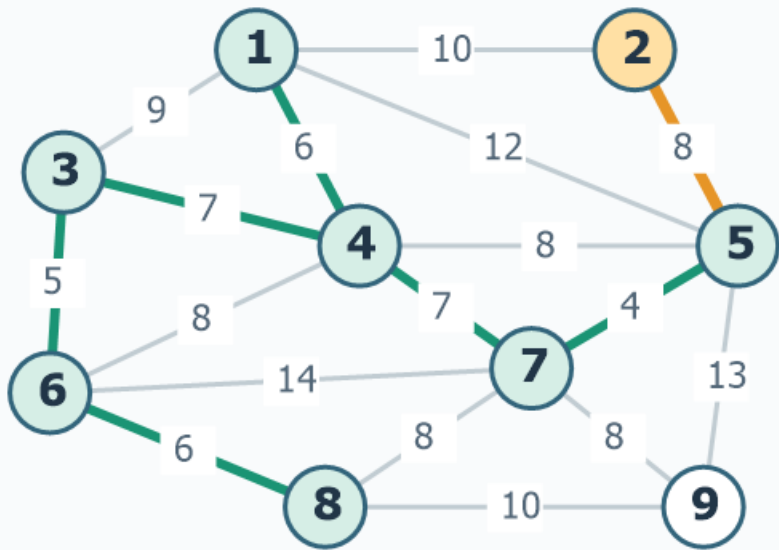


เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 3, 4, 5, 6, 7, 8\}$
น้ำหนักสะสม = 35

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

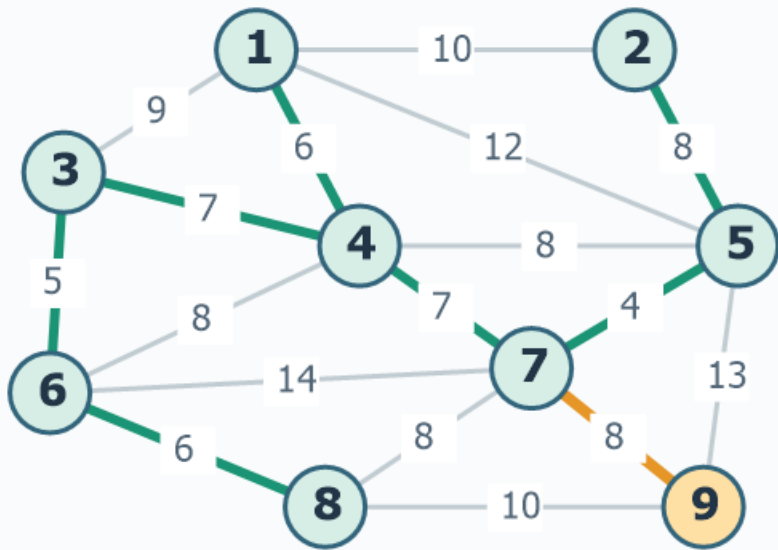
เริ่ม $S = \{1\}$ | เส้นสีส้ม = เส้นที่เพิ่มรอบนี้ | เส้นสีเขียว = เส้นที่เลือกก่อนหน้านี้

รอบที่ 7: เลือกเส้น 5-2 น้ำหนัก 8



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 2, 3, 4, 5, 6, 7, 8\}$
น้ำหนักสะสม = 43

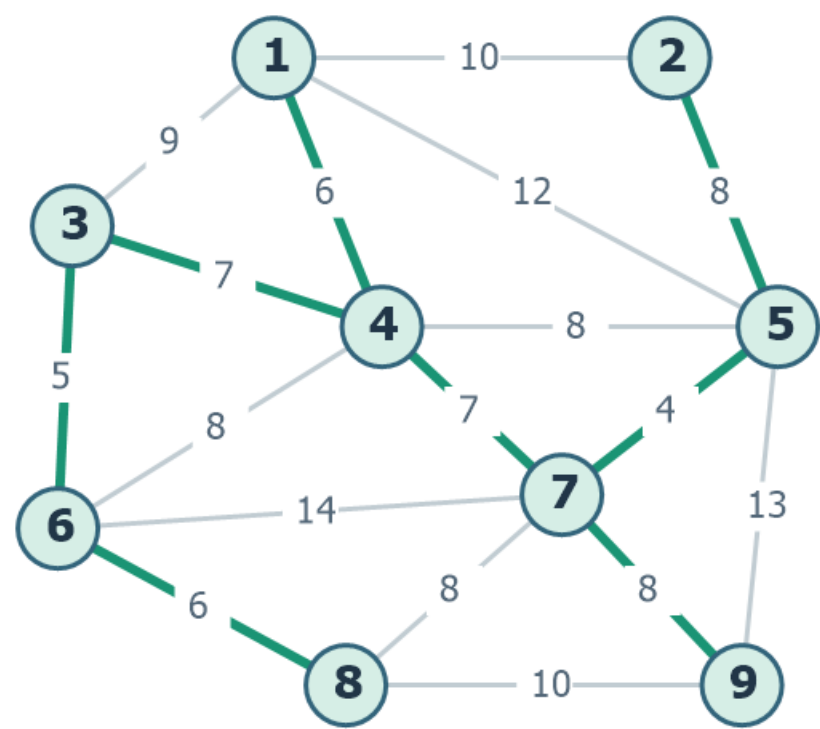
รอบที่ 8: เลือกเส้น 7-9 น้ำหนัก 8



เลือกเส้นเบาสุดที่เชื่อมออกนอกกลุ่ม S
 $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
น้ำหนักสะสม = 51

กราฟที่ 2 | Prim: สรุปร MST

เริ่ม $S = \{1\}$ | น้ำหนักเท่ากันเลือกคู่ปลายเส้นที่มีเลขน้อยก่อน | ไม่เลือกเส้นที่ทำให้เกิดวงจร



รอบ	เส้นที่เลือก	น้ำหนัก
1	1-4	6
2	4-3	7
3	3-6	5
4	6-8	6
5	4-7	7
6	7-5	4
7	5-2	8
8	7-9	8

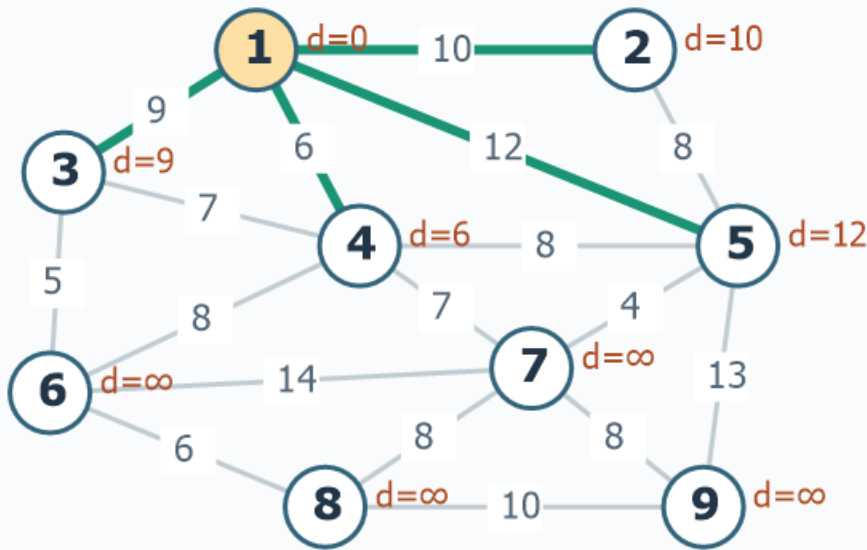
น้ำหนักรวม = 6 + 7 + 5 + 6 + 7 + 4 + 8 + 8 = 51

มี 9 จุด และ 8 เส้น เชื่อมครบทุกจุดโดยไม่มีวงจร

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

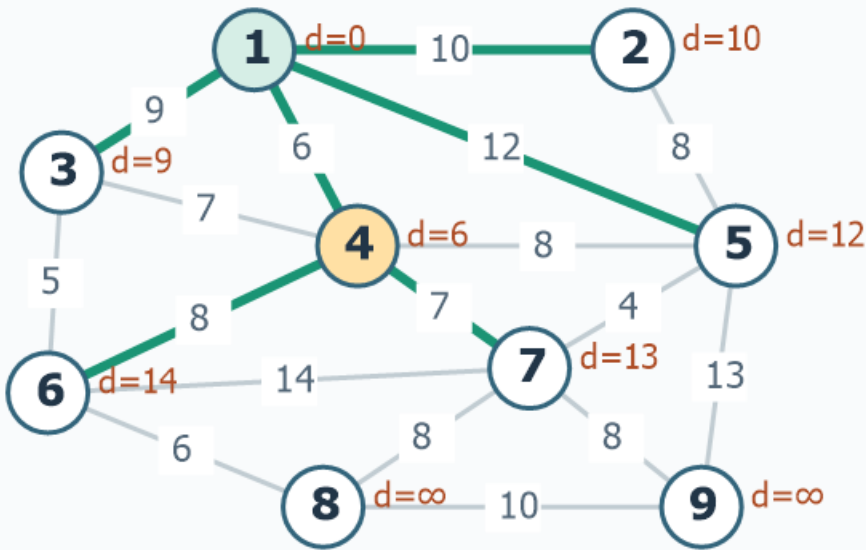
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 1: ยืนยันจุด 1, $d(1) = 0$



- $d(2): \infty \rightarrow 0 + 10 = 10$
- $d(3): \infty \rightarrow 0 + 9 = 9$
- $d(4): \infty \rightarrow 0 + 6 = 6$
- $d(5): \infty \rightarrow 0 + 12 = 12$

รอบที่ 2: ยืนยันจุด 4, $d(4) = 6$

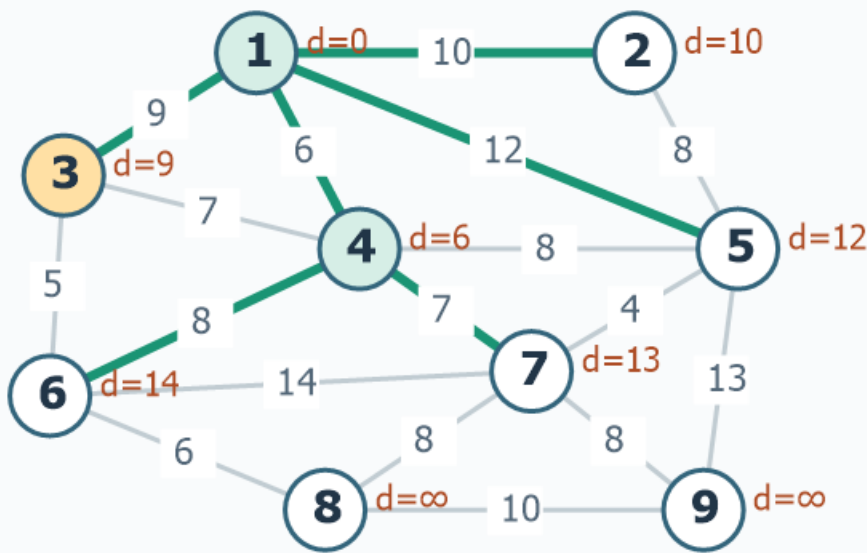


- $d(6): \infty \rightarrow 6 + 8 = 14$
- $d(7): \infty \rightarrow 6 + 7 = 13$

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

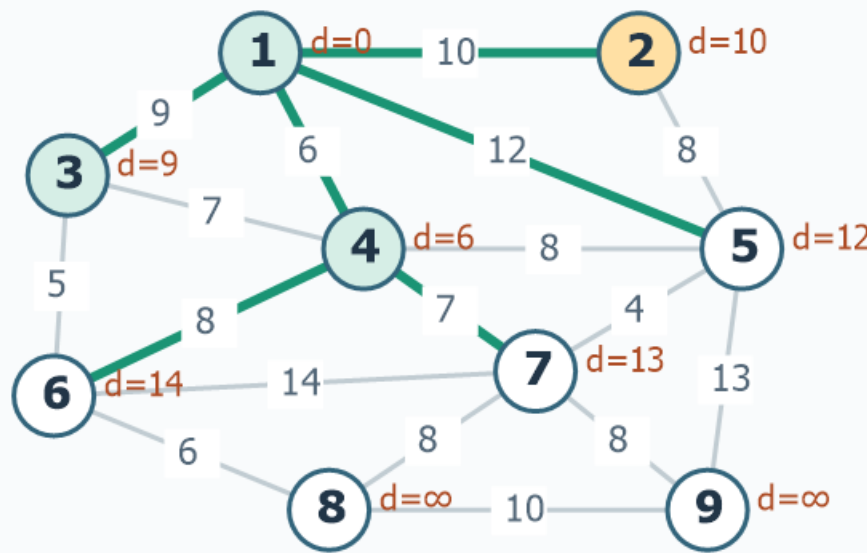
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 3: ยืนยันจุด 3, $d(3) = 9$



ไม่มีระยะทางที่ลดลงในรอบนี้

รอบที่ 4: ยืนยันจุด 2, $d(2) = 10$

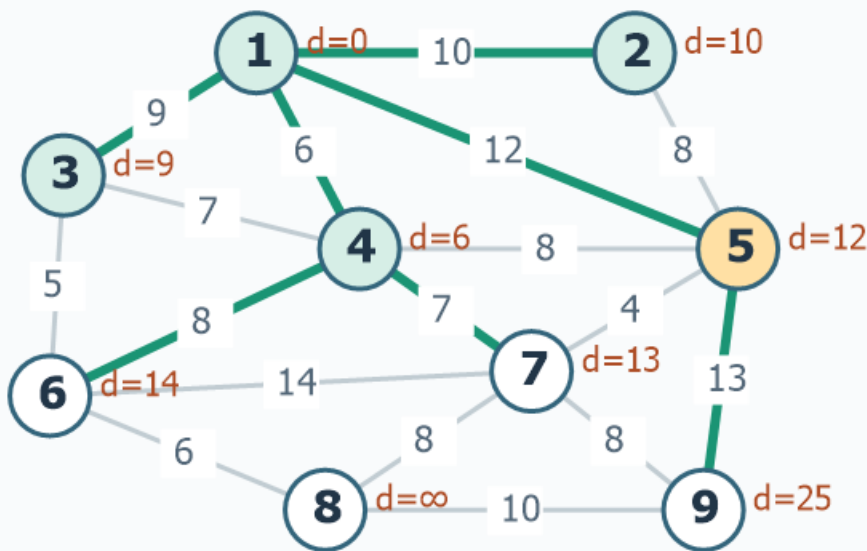


ไม่มีระยะทางที่ลดลงในรอบนี้

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

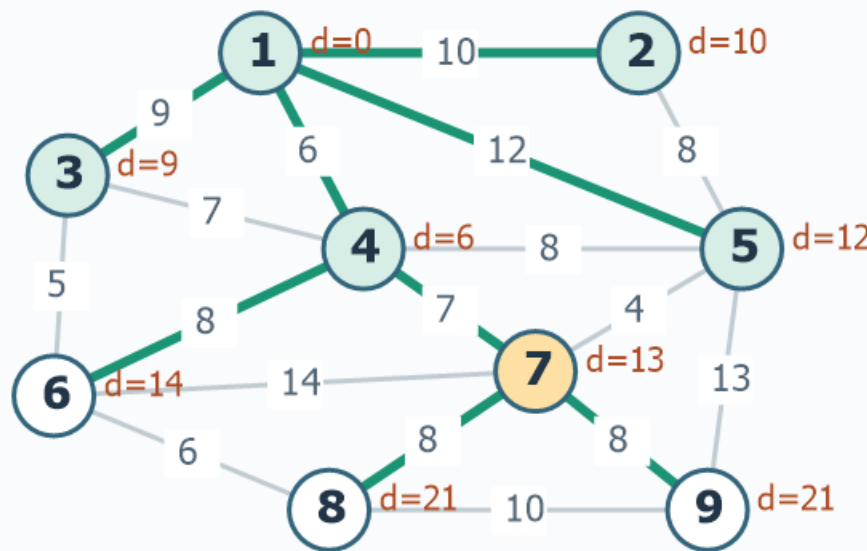
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 5: ยืนยันจุด 5, $d(5) = 12$



$d(9): \infty \rightarrow 12 + 13 = 25$

รอบที่ 6: ยืนยันจุด 7, $d(7) = 13$



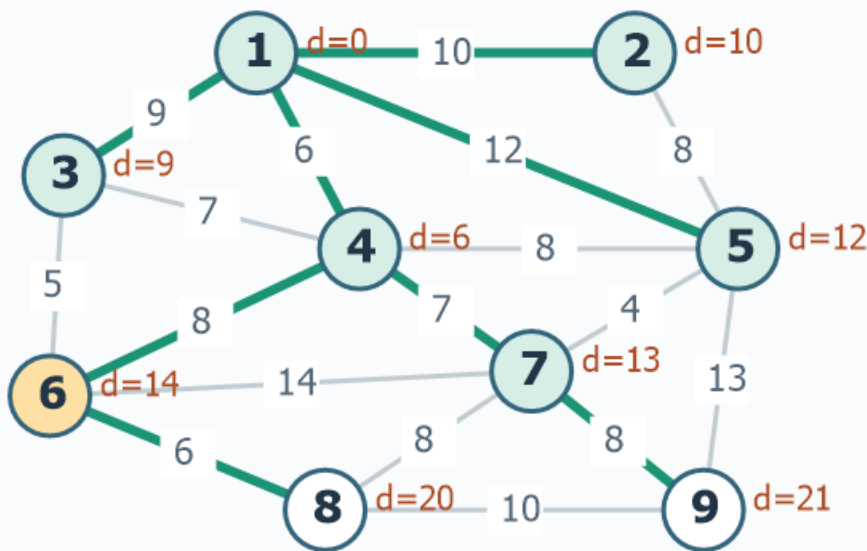
$d(8): \infty \rightarrow 13 + 8 = 21$

$d(9): 25 \rightarrow 13 + 8 = 21$

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

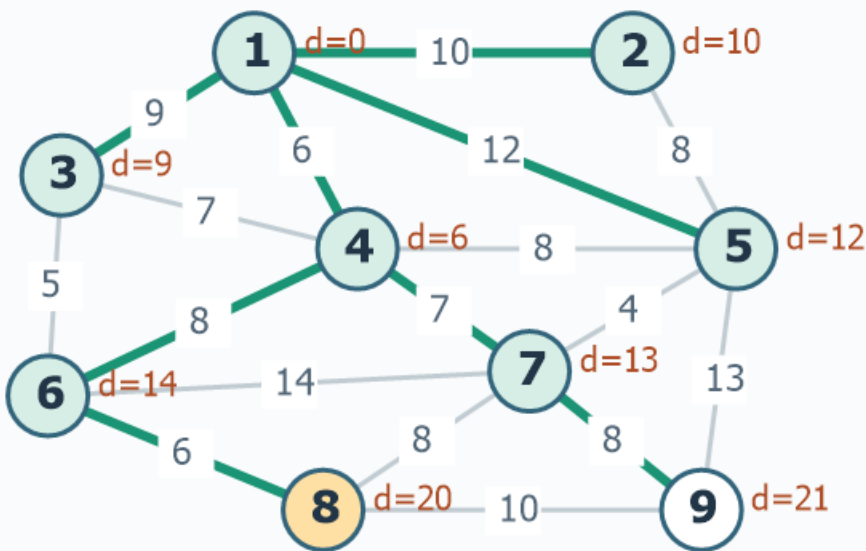
เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 7: ยืนยันจุด 6, $d(6) = 14$



$d(8): 21 \rightarrow 14 + 6 = 20$

รอบที่ 8: ยืนยันจุด 8, $d(8) = 20$

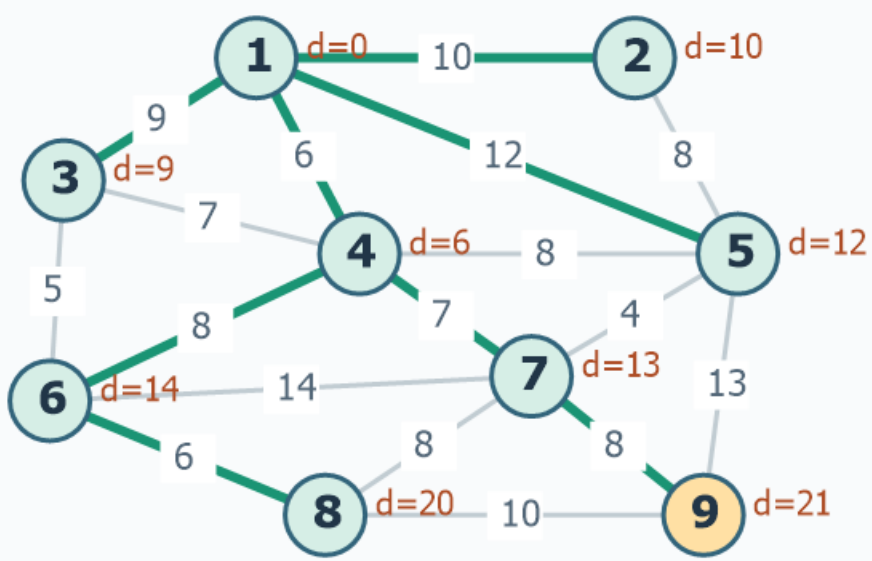


ไม่มีระยะทางที่ลดลงในรอบนี้

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

เริ่ม $d(1) = 0$, จุดอื่น = ∞ | สีส้ม = ยืนยันรอบนี้ | เส้นสีเขียว = ต้นไม้เส้นทางชั่วคราว

รอบที่ 9: ยืนยันจุด 9, $d(9) = 21$



ไม่มีระยะทางที่ลดลงในรอบนี้

กราฟที่ 2 | Dijkstra: ตารางระยะทางแต่ละรอบ

เริ่ม $d(1) = 0$, จุดอื่น $= \infty$ | ยืนยันจุดที่มีระยะทางน้อยสุด | ค่าเท่ากันเลือกจุดเลขน้อยก่อน

รอบ	ยืนยัน	d(1)	d(2)	d(3)	d(4)	d(5)	d(6)	d(7)	d(8)	d(9)
1	1	0	10	9	6	12	∞	∞	∞	∞
2	4	0	10	9	6	12	14	13	∞	∞
3	3	0	10	9	6	12	14	13	∞	∞
4	2	0	10	9	6	12	14	13	∞	∞
5	5	0	10	9	6	12	14	13	∞	25
6	7	0	10	9	6	12	14	13	21	21
7	6	0	10	9	6	12	14	13	20	21
8	8	0	10	9	6	12	14	13	20	21
9	9	0	10	9	6	12	14	13	20	21

ปรับระยะทางเมื่อ $d(\text{จุดปัจจุบัน}) + \text{น้ำหนักเส้น} < \text{ระยะทางเดิมเท่านั้น}$

กราฟที่ 2 | Dijkstra: เส้นทางสั้นที่สุดจากจุด 1

เมื่อระยะทางใหม่เท่าค่าเดิม ให้เก็บจุดก่อนหน้าเดิมไว้ จึงได้คำตอบตามเส้นทางด้านล่าง

ปลายทาง	เส้นทางจากจุด 1	ระยะทาง
1	1	0
2	1 → 2	10
3	1 → 3	9
4	1 → 4	6
5	1 → 5	12
6	1 → 4 → 6	14
7	1 → 4 → 7	13
8	1 → 4 → 6 → 8	20
9	1 → 4 → 7 → 9	21